

User Manual

for S32K1_S32M24X CRC Driver

Document Number: UM2CRCASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	7
3.4 Deviations from Requirements	7
3.5 Driver Limitations	9
3.6 Driver usage and configuration tips	9
3.6.1 CRC Calculation Type	9
3.6.1.1 Calculation Type supported by CRC_IP_LOOKUP_TABLES_CALCULATION	9
3.6.1.2 Calculation Type supported by CRC_IP_SOFTWARE_CALCULATION	9
3.6.1.3 Calculation Type supported by CRC_IP_HARDWARE_CALCULATION	10
3.6.1.4 Calculation Type supported by CRC_IP_LOOKUP_TABLES_CALCULATION with Select Algorithm for table calculate function	10
3.6.2 CRC Initialization.	10
3.6.3 CRC Channel Configuration.	10
3.6.4 CRC Channel Calculation.	10
3.6.5 CRC Channel Get Result.	10
3.6.6 CRC Examples for using the driver	10
3.6.6.1 Example Calculate Autosar CRC16 CCITT.	11
3.6.6.2 Example Calculate Autosar CRC32 ETHERNET.	13
3.6.6.3 Example Calculate CRC CUSTOM.	14
3.6.6.4 Example Calculate CRC with DMA support (only support CRC_IP_HARDWARE← _CALCULATION).	16
3.6.6.5 Example Calculate CRC with Select Algorithm for table (only support CRC_IP_T← ABLE_CALCULATION with CRC_PROTOCOL_32BIT_CASTAGNOLI).	18
3.7 Runtime errors	19
3.8 Symbolic Names Disclaimer	21
3.9 Usage in non AUTOSAR context	21
4 Configuration Parameters	23
4.1 Crc	24
4.1.1 Module Crc	25
4.1.2 Container CrcGeneral	25
4.1.3 Parameter CrcDetectError	26
4.1.4 Parameter CrcEnableUserModeSupport	26
4.1.5 Parameter CrcDmaSupportEnable	27
4.1.6 Parameter CrcSelectAlgorithmSupportEnable	27
4.1.7 Parameter CrcMultiPartitionEnable	27
4.1.8 Parameter CrcVersionInfoApi	28
4.1.9 Parameter Crc8Mode	28
4.1.10 Parameter Crc8H2FMode	29
4.1.11 Parameter Crc16Mode	29
4.1.12 Parameter Crc16ARCMode	30
4.1.13 Parameter Crc32Mode	30
4.1.14 Parameter Crc32P4Mode	30

4.1.15	Parameter	Crc32CMode	31
4.1.16	Parameter	Crc64Mode	31
4.1.17	Container	CrcChannelConfig	32
4.1.18	Parameter	CrcLogicChannelName	32
4.1.19	Parameter	CrcAutosarSelect	32
4.1.20	Parameter	CrcCalculationType	33
4.1.21	Reference	CrcPartitionRefOfChannel	33
4.1.22	Container	CrcHardwareConfig	34
4.1.23	Parameter	CrcHwInstance	34
4.1.24	Parameter	CrcHwChannel	35
4.1.25	Parameter	CrcDmaChannelEnable	35
4.1.26	Parameter	CrcDmaTransferSize	35
4.1.27	Parameter	CrcPolynomialSelectionExtendEnable	36
4.1.28	Reference	CrcDmaLogicChannelName	36
4.1.29	Container	CrcProtocolInfo	37
4.1.30	Parameter	CrcProtocolType	37
4.1.31	Parameter	CrcAlgorithmType	37
4.1.32	Parameter	CrcPolynomialValue	38
4.1.33	Parameter	CrcWriteBitSwap	38
4.1.34	Parameter	CrcWriteByteSwap	39
4.1.35	Parameter	CrcReadBitSwap	39
4.1.36	Parameter	CrcReadByteSwap	40
4.1.37	Parameter	CrcInversionEnable	40
4.1.38	Container	CrcEcucPartitionRefArray	40
4.1.39	Reference	CrcEcucPartitionRef	41
4.1.40	Container	CommonPublishedInformation	41
4.1.41	Parameter	ArReleaseMajorVersion	41
4.1.42	Parameter	ArReleaseMinorVersion	42
4.1.43	Parameter	ArReleaseRevisionVersion	42
4.1.44	Parameter	ModuleId	43
4.1.45	Parameter	SwMajorVersion	43
4.1.46	Parameter	SwMinorVersion	43
4.1.47	Parameter	SwPatchVersion	44
4.1.48	Parameter	VendorApiInfix	44
4.1.49	Parameter	VendorId	45
4.2	CRC_IP		45
4.2.1	Module	Crc_Ip	46
5	Module Index		47
5.1	Software Specification		47
6	Module Documentation		48
6.1	CRC HLD Driver		48
6.1.1	Detailed Description		48
6.1.2	Data Structure Documentation		50
6.1.2.1	struct Crc_PartitionType		50
6.1.2.2	struct Crc_InitType		50
6.1.3	Macro Definition Documentation		50
6.1.3.1	CDD_CRC_MODULE_ID		50
6.1.3.2	CRC_INSTANCE_ID		51
6.1.3.3	CRC_CALCULATECRC8_ID		51
6.1.3.4	CRC_CALCULATECRC16_ID		51
6.1.3.5	CRC_CALCULATECRC32_ID		51
6.1.3.6	CRC_GETVERSIONINFO_ID		51

6.1.3.7	CRC_CALCULATECRC8H2F_ID	52
6.1.3.8	CRC_CALCULATECRC32P4_ID	52
6.1.3.9	CRC_CALCULATECRC64_ID	52
6.1.3.10	CRC_CALCULATECRC16ARC_ID	52
6.1.3.11	CRC_INIT_ID	52
6.1.3.12	CRC_SETCHANNELCONFIG_ID	53
6.1.3.13	CRC_SETCHANNELCALCULATE_ID	53
6.1.3.14	CRC_GETCHANNELRESULT_ID	53
6.1.3.15	CRC_CALCULATECRC32C_ID	53
6.1.3.16	CRC_E_INVALID_CHANNEL	53
6.1.3.17	CRC_E_INVALID_POINTER	54
6.1.3.18	CRC_E_PARAM_CONFIG	54
6.1.3.19	CRC_E_INIT_FAILED	54
6.1.3.20	CRC_TYPES_VENDOR_ID	54
6.1.4	Types Reference	54
6.1.4.1	Crc_ChannelConfigType	54
6.1.4.2	Crc_ProtocolType	55
6.1.5	Function Reference	55
6.1.5.1	Crc_Init()	55
6.1.5.2	Crc_SetChannelConfig()	55
6.1.5.3	Crc_SetChannelCalculate()	55
6.1.5.4	Crc_GetChannelResult()	56
6.1.5.5	Crc_CalculateCRC8()	56
6.1.5.6	Crc_CalculateCRC8H2F()	57
6.1.5.7	Crc_CalculateCRC16()	57
6.1.5.8	Crc_CalculateCRC16ARC()	58
6.1.5.9	Crc_CalculateCRC32()	58
6.1.5.10	Crc_CalculateCRC32P4()	59
6.1.5.11	Crc_CalculateCRC64()	60
6.1.5.12	Crc_CalculateCRC32C()	60
6.1.5.13	Crc_GetVersionInfo()	61
6.2	CRC IPL Driver	62
6.2.1	Detailed Description	62
6.2.2	Data Structure Documentation	62
6.2.2.1	struct Crc_Ip_CrcProtocolInfoType	62
6.2.2.2	struct Crc_Ip_LogicChannelStateType	63
6.2.2.3	struct Crc_Ip_LogicChannelConfigType	63
6.2.2.4	struct Crc_Ip_LogicChannelType	63
6.2.2.5	struct Crc_Ip_InitType	64
6.2.3	Enum Reference	64
6.2.3.1	Crc_Ip_CrcWidthType	64
6.2.3.2	Crc_Ip_CalculationType	64
6.2.3.3	Crc_Ip_ProtocolType	64
6.2.3.4	Crc_Ip_AlgorithmType	65
6.2.4	Function Reference	65
6.2.4.1	Crc_Ip_Init()	65
6.2.4.2	Crc_Ip_SetChannelConfig()	66
6.2.4.3	Crc_Ip_SetChannelCalculate()	66
6.2.4.4	Crc_Ip_GetChannelResult()	67

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR CRC for S32K1 and S32M24x. AUTOSAR CRC driver configuration parameters and deviations from the specification are described in Driver chapter of this document. A↔UTOSAR CRC driver requirements and APIs are described in the AUTOSAR CRC driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64

- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release R21-11
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

For CDD: CRC is a both Complex Device Driver (CDD) and support AUTOSAR requirements regarding this module. It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

- The CRC driver is implemented as an complex device driver. It uses the CRC hardware peripheral which provides support for implementing the CRC calculations.
- The driver offers: Software Calculation, Lookup Table Calculation and Hardware Calculation API to the upper layer that can be used to configure the CRC and initiate CRC calculations. On Lookup Table Calculate, the driver supports not only bitwise algorithm but also slice-by-4 and slice-by-8 algorithms with CRC_PROT↔OCOL_32BIT_CASTAGNOLI protocol. The slice-by-N algorithm calculates CRC faster than bitwise but requires more RAM.
- Hardware and software settings can be configured using an Autosar standard configuration tool. The information required for a CRC data calculation will be configured in a data structure that will be sent as parameter to the API of the driver.

3.3 Hardware Resources

The CRC Driver consists of:

1. CRC IP

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR CRC Driver software specification in some places. The table below identifies the AUTOSAR requirements that are not implemented or out of scope for the CRC Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, not implemented or out of scope for the CRC driver.

Requirement	Status	Description	Notes
SWS_Crc_00051	N/S	These requirements are not applicable to this specification.(SRS_BSW_00344, SRS_BSW_00404, SRS_BSW_00405, SRS_BSW_00170, SRS_BSW_00383, SRS_BSW_00384, SRS_BSW_00388, SRS_BSW_00389, SRS_BSW_00395, SRS_BSW_00398, SRS_BSW_00399, SRS_BSW_00400, SRS_BSW_00401, SRS_BSW_00375, SRS_BSW_00101, SRS_BSW_00416, SRS_BSW_00406, SRS_BSW_00168, SRS_BSW_00423, SRS_BSW_00424, SRS_BSW_00425, SRS_BSW_00427, SRS_BSW_00428, SRS_BSW_00429, SRS_BSW_00432, SRS_BSW_00433, SRS_BSW_00336, SRS_BSW_00337, SRS_BSW_00369, SRS_BSW_00339, SRS_BSW_00422, SRS_BSW_00417, SRS_BSW_00323, SRS_BSW_00409, SRS_BSW_00385, SRS_BSW_00386, SRS_BSW_00161, SRS_BSW_00162, SRS_BSW_00005, SRS_BSW_00415, SRS_BSW_00164, SRS_BSW_00325, SRS_BSW_00342, SRS_BSW_00343, SRS_BSW_00160, SRS_BSW_00007, SRS_BSW_00347, SRS_BSW_00305, SRS_BSW_00307, SRS_BSW_00373, SRS_BSW_00327, SRS_BSW_00335, SRS_BSW_00350, SRS_BSW_00410, SRS_BSW_00314, SRS_BSW_00348, SRS_BSW_00353, SRS_BSW_00361, SRS_BSW_00302, SRS_BSW_00328, SRS_BSW_00312, SRS_BSW_00006, SRS_BSW_00304, SRS_BSW_00378, SRS_BSW_00306, SRS_BSW_00308, SRS_BSW_00309, SRS_BSW_00358, SRS_BSW_00414, SRS_BSW_00359, SRS_BSW_00360, SRS_BSW_00330, SRS_BSW_00331, SRS_BSW_00009, SRS_BSW_00172, SRS_BSW_00010, SRS_BSW_00333, SRS_BSW_00321, SRS_BSW_00341, SRS_BSW_00334, SRS_LIBS_00001, SRS_LIBS_00002, SRS_LIBS_00003, SRS_LIBS_00004, SRS_LIBS_00007, SRS_LIBS_00008, SRS_LIBS_00010, SRS_LIBS_00012, SRS_LIBS_00013, SRS_LIBS_00015, SRS_LIBS_00016, SRS_LIBS_00017)	Not a requirement.

Requirement	Status	Description	Notes
SWS_Crc_00065	N/S	If the CRC calculation within the function Crc_CalculateCRC64 is performed by hardware, then the CRC module's implementer shall ensure reentrancy of this function by implementing a (software based) locking mechanism	There are no platforms supporting Crc64 Hardware calculation (Hardware limitation).

3.5 Driver Limitations

The CRC Driver has the following limitations:

- Post Build Compiler not implemented.
- For each CRC type, the user can configure only one of the three methods: Hardware, Look-up-Table, Runtime(↔ Software).
- The logic channel with "Autosar Select: NON_AUTOSAR" is not supported the "Multipartition feature".
- Crc64Mode hardware not supported.

3.6 Driver usage and configuration tips

This Driver is a Complex Device Driver. Complete driver functionality together with API description can be found below.

3.6.1 CRC Calculation Type

- When Calculation Type is CRC_IP_LOOKUP_TABLES_CALCULATION or CRC_IP_HARDWARE_↔ CALCULATION, the Protocol shall be selected from the supported ones.
- Autosar Library supports only the Autosar Protocols.
- Polynomial configuration is enabled when the following Protocols are selected: CRC_PROTOCOL_8BIT, CRC_PROTOCOL_16BIT, CRC_PROTOCOL_32BIT, CRC_PROTOCOL_64BIT.
- Note: Each Calculation Type supports a specific list of Protocols. If out of range there will be an error message.

3.6.1.1 Calculation Type supported by CRC_IP_LOOKUP_TABLES_CALCULATION

- CRC_PROTOCOL_8BIT_SAE_J1850
- CRC_PROTOCOL_8BIT_H2F
- CRC_PROTOCOL_16BIT_CCITT
- CRC_PROTOCOL_16BIT_ARC
- CRC_PROTOCOL_32BIT_ETHERNET
- CRC_PROTOCOL_32BIT_E2EP4
- CRC_PROTOCOL_32BIT_CASTAGNOLI
- CRC_PROTOCOL_64BIT_ECMA

3.6.1.2 Calculation Type supported by CRC_IP_SOFTWARE_CALCULATION

- CRC_PROTOCOL_8BIT_CUSTOM
- CRC_PROTOCOL_16BIT_CUSTOM
- CRC_PROTOCOL_32BIT_CUSTOM

- CRC_PROTOCOL_64BIT_CUSTOM
- CRC_PROTOCOL_8BIT_SAE_J1850
- CRC_PROTOCOL_8BIT_H2F
- CRC_PROTOCOL_16BIT_CCITT
- CRC_PROTOCOL_16BIT_ARC
- CRC_PROTOCOL_32BIT_ETHERNET
- CRC_PROTOCOL_32BIT_CASTAGNOLI
- CRC_PROTOCOL_32BIT_E2EP4
- CRC_PROTOCOL_64BIT_ECMA

3.6.1.3 Calculation Type supported by CRC_IP_HARDWARE_CALCULATION

- CRC_PROTOCOL_16BIT_CUSTOM
- CRC_PROTOCOL_32BIT_CUSTOM
- CRC_PROTOCOL_16BIT_CCITT
- CRC_PROTOCOL_32BIT_ETHERNET

3.6.1.4 Calculation Type supported by CRC_IP_LOOKUP_TABLES_CALCULATION with Select Algorithm for table calculate function

- CRC_PROTOCOL_32BIT_CASTAGNOLI

3.6.2 CRC Initialization.

- The [Crc_Init\(\)](#) function shall initialize the CRC hardware peripheral(s) and the internal driver context, according to the input configuration data. The application shall ensure that the [Crc_Init\(\)](#) function is called first. Only the [Crc_GetVersionInfo\(\)](#) can be called before [Crc_Init\(\)](#).

3.6.3 CRC Channel Configuration.

- The function receives pointer to a configuration structure that shall be loaded into the Logic Channel. According to input parameters this function configures channel's CRC width, polynomial and whether Swap and/or Inversion functionality is applied on CRC input data and/or signature or not.

3.6.4 CRC Channel Calculation.

- The CRC driver provides the function of calculating the CRC of a data stream of a certain length. The CRC calculation uses the CRC width, polynomial, and previously set swap/inversion options for the channel using the [Crc_SetChannelConfig\(\)](#) function. The CRC is computed on an input vector data pointed by pCrcData and has a length set to CrcLength. The function synchronously calculates the CRC and returns the resulting CRC.

3.6.5 CRC Channel Get Result.

- Driver provides functionality for reading CRC result from required channel. The [Crc_GetChannelResult\(\)](#) function reads CRC Results (CRC checksum).

3.6.6 CRC Examples for using the driver

- This sub-chapter lists some examples of how to use the CRC Driver.

3.6.6.1 Example Calculate Autosar CRC16 CCITT.

To compute CRC checksum according to CCITT-FALSE CRC16 standard, please follow the steps below:

1. Open EB Tresos then create New Configuration Project
2. In Tresos GUI, add CRC Configuration in tab "Module Configuration"

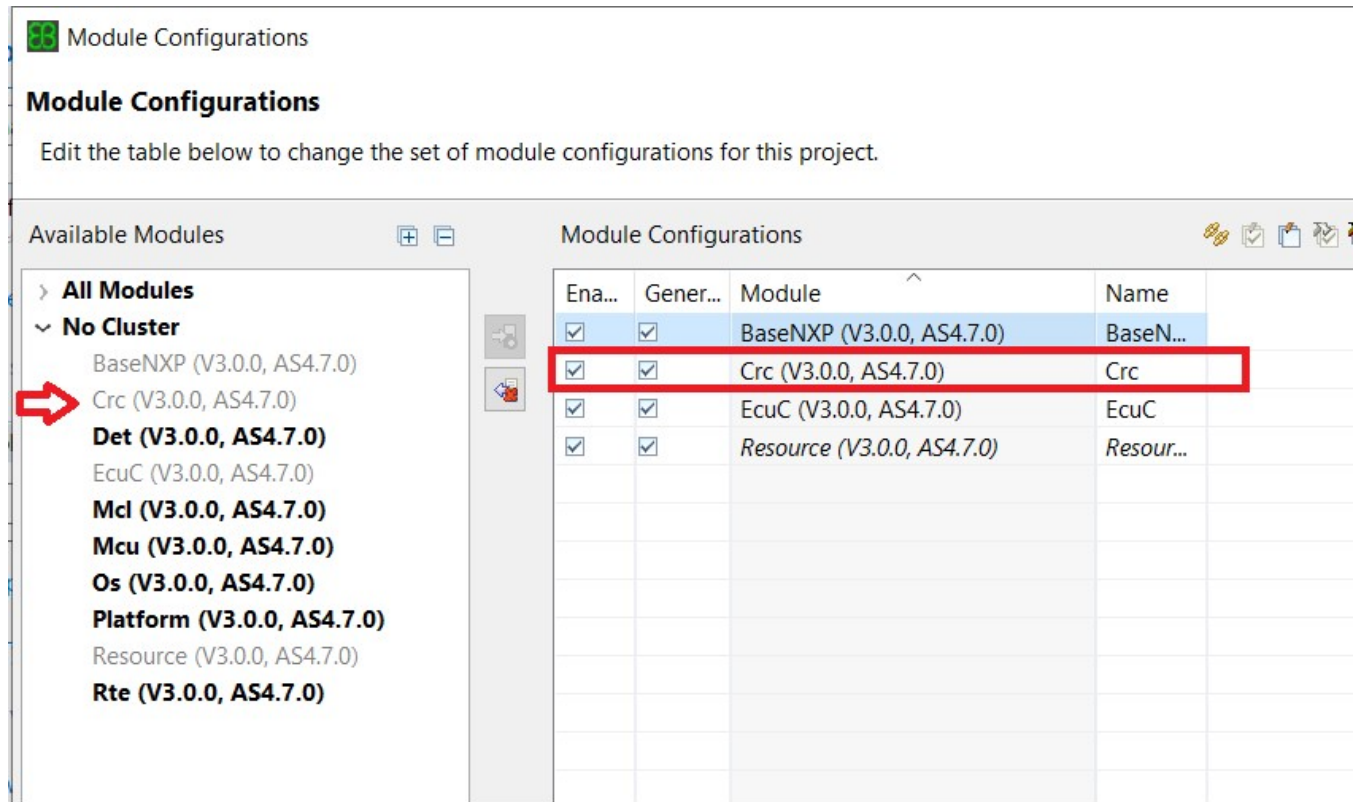


Figure 3.1 CRC add Configuration

3. In "General", enable "CRC16Mode" and select "CRC_16_HARDWARE"

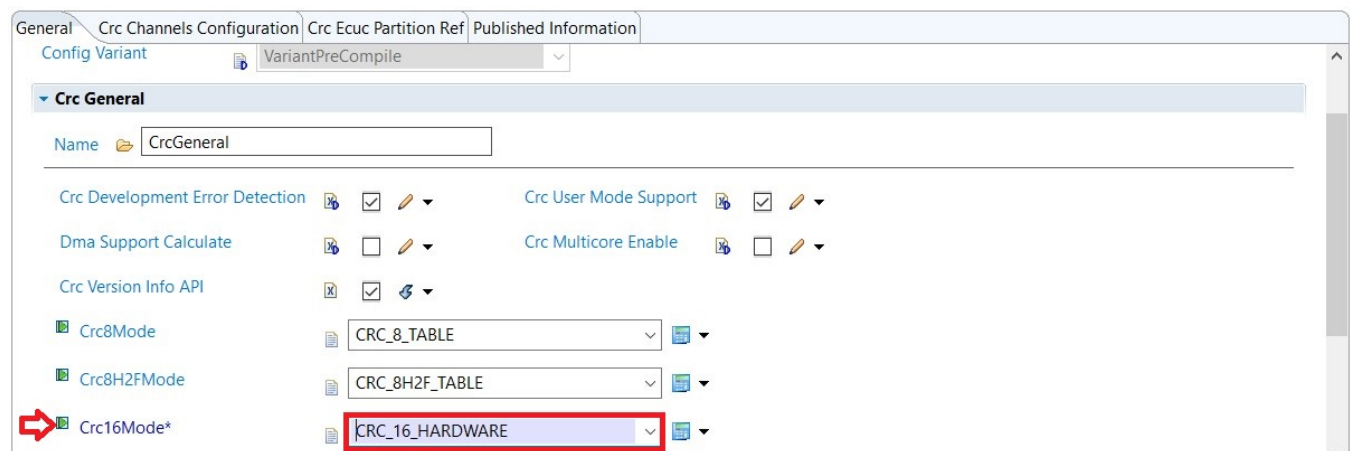


Figure 3.2 CRC enable AUTOSAR 16 CCITT

- In "CRC Channel Configuration", add new CRC logic channel with default config.

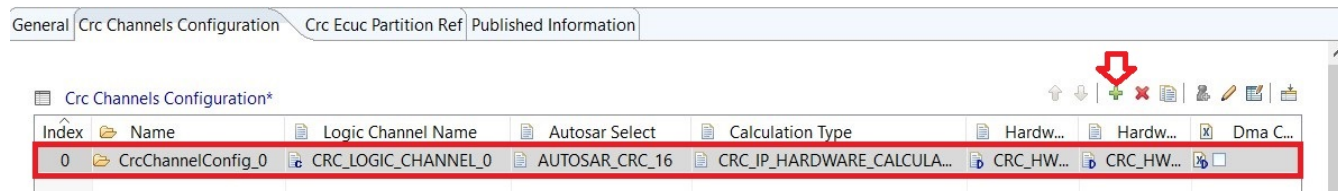


Figure 3.3 CRC add logic Channel with default config

- In CRC logic channel configuration, using the following values for the attributes.
 - Set "Calculates Type" to CRC_IP_HARDWARE_CALCULATION
 - Set "Autosar Select" to AUTOSAR_CRC_16.

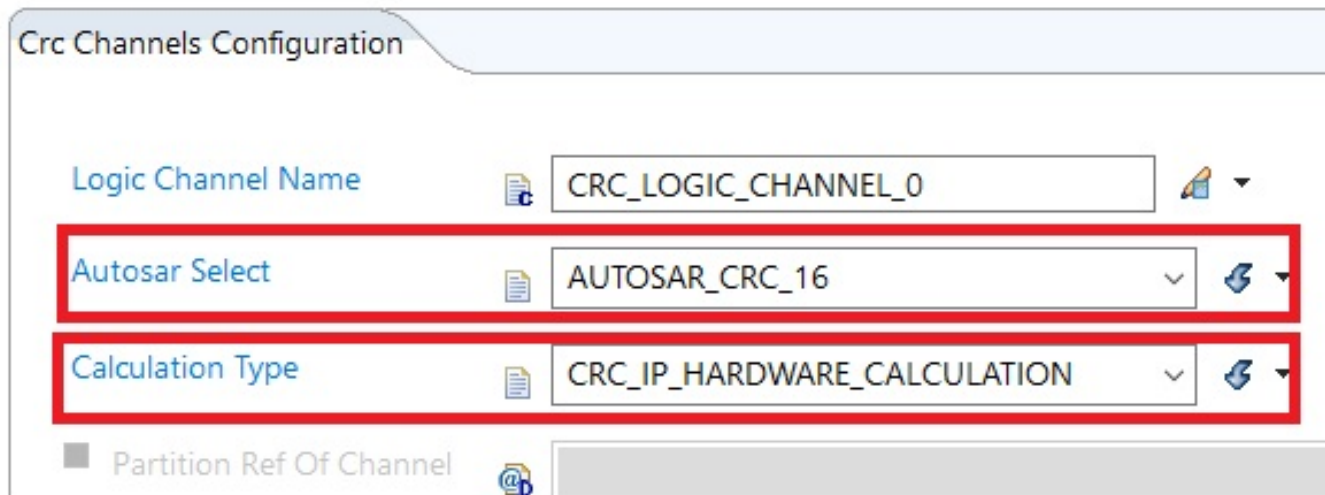


Figure 3.4 CRC config Calculates Type and Autosar Select

- Set "Protocol Type" to CRC_PROTOCOL_16BIT_CCITT_FALSE.

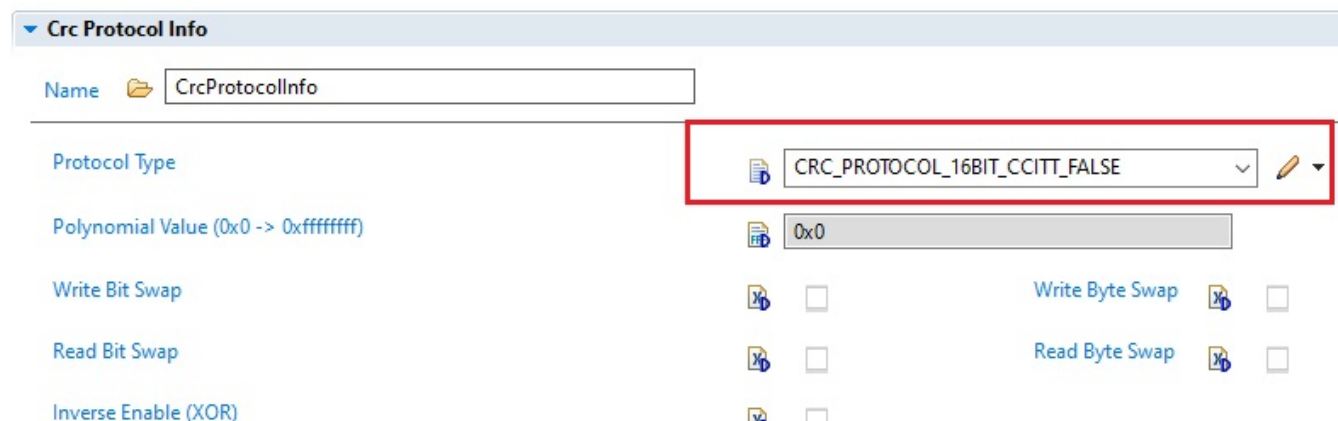


Figure 3.5 CRC config protocol is 16BIT_CCITT_FALSE

6. Generate the configuration files from Tresos without errors or warnings.

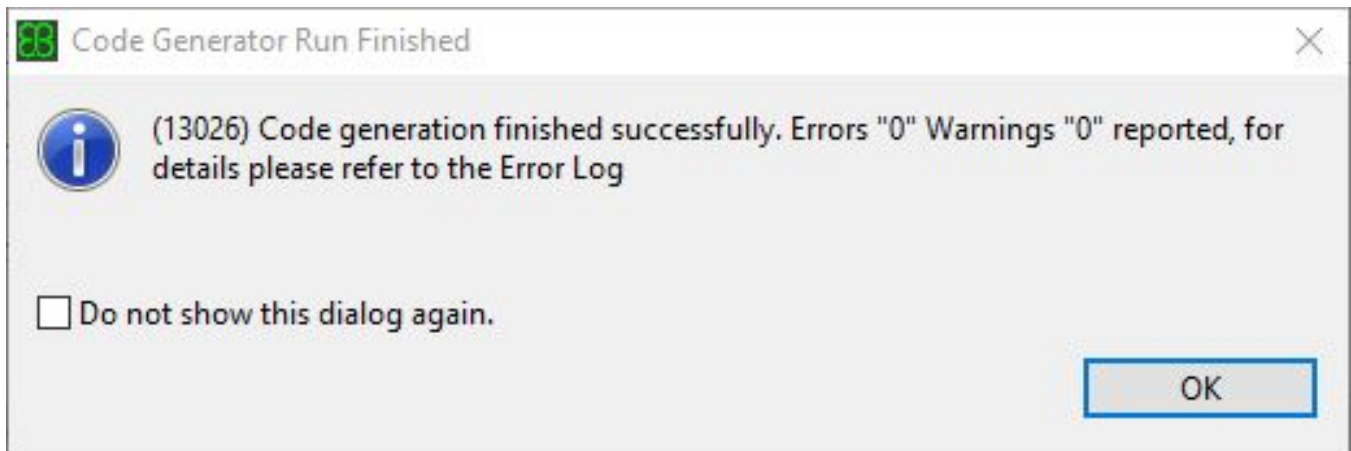


Figure 3.6 Status Generate Config

7. Write the application code, following the steps.

- Call the function `Crc_Init()` providing it as parameter a NULL pointer.
- Call the function `Crc_Calculate_CRC16()` to start calculation. This function will use the pre-configured Logic Channels in Auto Mode. After the calculation is completed, it will return the results

```
/* Initialize CRC driver */
Crc_Init(NULL_PTR);
/* Start calculate then get result */
result = Crc_CalculateCRC16((const uint8_t *)CRC_data, CRC_DATA_SIZE, 0xFFFF, true);
```

Figure 3.7 CRC application code

3.6.6.2 Example Calculate Autosar CRC32 ETHERNET.

To compute CRC checksum according to IEEE 802.3 Ethernet standard, please follow the steps below:

1. Please follow the steps 1 to 4 from "Example Calculate Autosar CRC16 CCITT"
2. In CRC logic channel configuration, using the following values for the attributes.
 - Set "Calculates Type" to `CRC_IP_HARDWARE_CALCULATION`
 - Set "Autosar Select" to `AUTOSAR_CRC_32`.

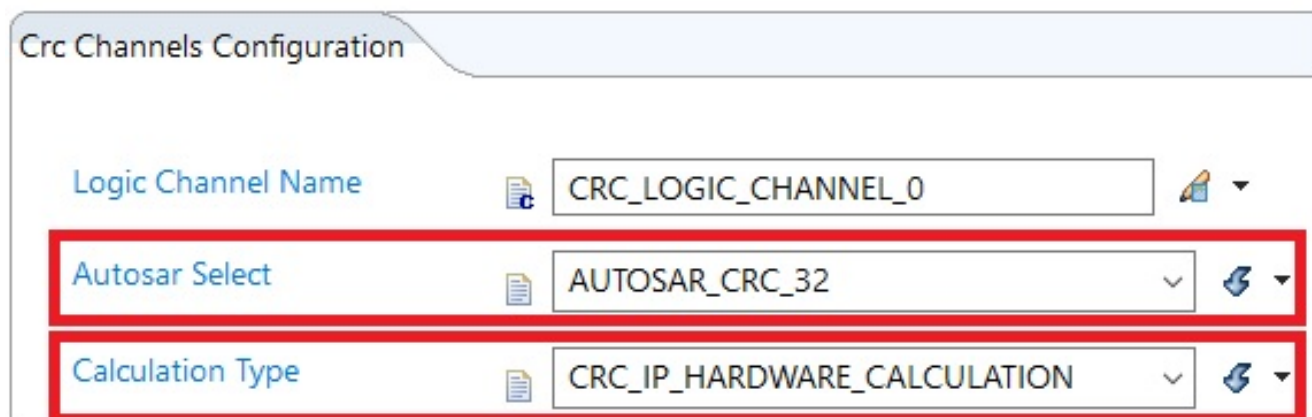


Figure 3.8 CRC config CRC config Calculates Type and Enable Autosar

- Set "Protocol Type" to CRC_PROTOCOL_32BIT_ETHERNET.

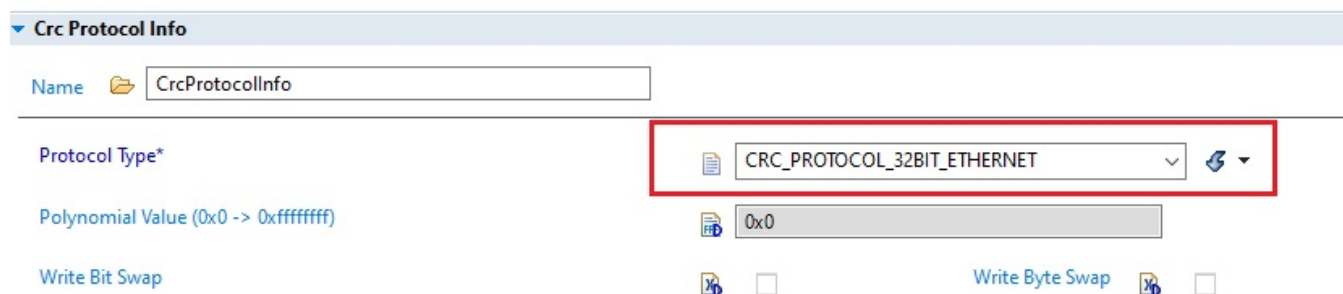


Figure 3.9 CRC config protocol is CRC_PROTOCOL_32BIT_ETHERNET

3. Generate the configuration files from Tresos without errors or warnings.
4. Write the application code, following the steps.
 - Call the function `Crc_Init()` providing it as parameter a NULL pointer.
 - Call the function `Crc_CalculateCRC32()` to start calculation. This function will use the pre-configured Logic Channels in Auto Mode. After the calculation is completed, it will return the results

```
/* Initialize CRC driver */
Crc_Init(NULL_PTR);
/* Start calculate then get result */
result = Crc_CalculateCRC32((const uint8_t *)CRC_data, CRC_DATA_SIZE, 0xFFFF, true);
```

Figure 3.10 CRC application code

3.6.6.3 Example Calculate CRC CUSTOM.

To compute any CRC CUSTOM Type is CRC_PROTOCOL_8BIT_CUSTOM (or CRC_PROTOCOL_16BIT_CUSTOM, CRC_PROTOCOL_32BIT_CUSTOM, CRC_PROTOCOL_64BIT_CUSTOM), please follow the

steps below:

1. Please follow the steps 1 to 4 from "Example Calculate Autosar CRC16 CCITT"
2. In CRC logic channel configuration, using the following values for the attributes.
 - Set "Calculates Type" to CRC_IP_RUNTIME_CALCULATION
 - Set "Autosar Select" to NON_AUTOSAR.

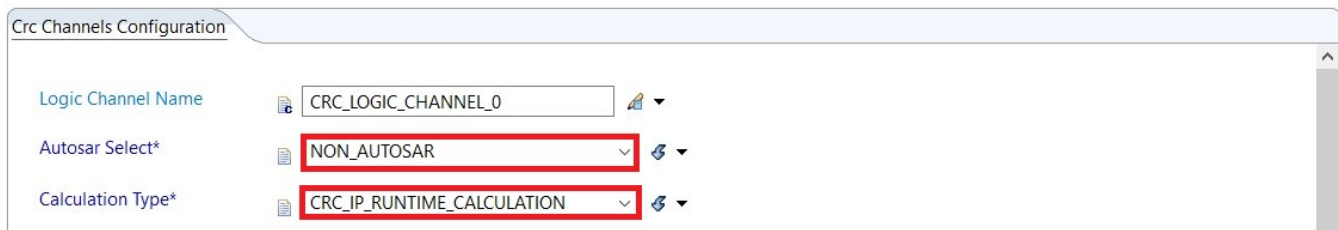


Figure 3.11 CRC Calculate Tye is CRC_IP_RUNTIME_CALCULATION

- Set "Protocol Type" to CRC_PROTOCOL_8BIT_CUSTOM and configure Polynomial Value, Swap Option and Inverse Enable

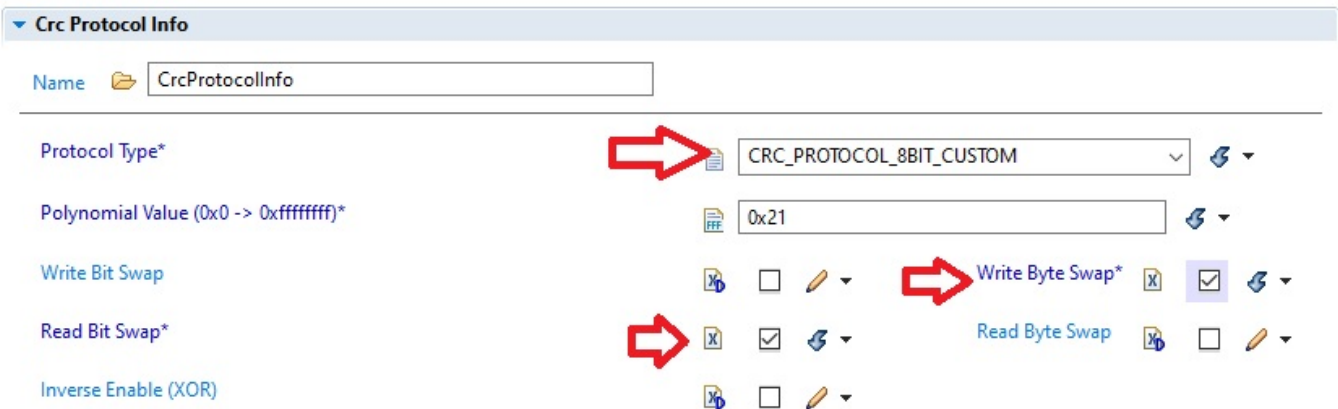


Figure 3.12 CRC config protocol CRC_PROTOCOL_8BIT_CUSTOM

3. Generate the configuration files from Tresos without errors or warnings.
4. Write the application code, following the steps.
 - Call the function `Crc_Init()` providing it as parameter a NULL pointer.
 - Call the function `Crc_SetChannelCalculate()` to start calculation. Calculation is based on User's logic channel configuration, Crc data path(pCrcData), Crc length (CrcLength) and start value (CrcStartValue). After the calculation is complete, this function will return the result

```

/* Initialize CRC driver */
Crc_Init(NULL_PTR);
/* Start calculate then get result */
result = Crc_SetChannelCalculate((const uint8_t *)CRC_data, CRC_DATA_SIZE, 0xFFFF, true);

```

Figure 3.13 CRC application code

3.6.6.4 Example Calculate CRC with DMA support (only support CRC_IP_HARDWARE_CALCULATION).

To compute CRC checksum use DMA support according to CCITT-FALSE CRC16 standard, please follow the steps below:

1. Please follow the steps 1 to 4 from "Example Calculate Autosar CRC16 CCITT"
2. In CRC logic channel configuration, using the following values for the attributes.
 - Note: Autosar Library Function does not support DMA
 - Set checkbox "DMA support Calculate" to Enable.



Figure 3.14 CRC DMA support Calculate Enable

- Set "Calculates Type" to CRC_IP_HARDWARE_CALCULATION
- "Autosar Select" must be NON_AUTOSAR, otherwise the "DMA Channel Enable" could not be selected.

Crc Channels Configuration

Logic Channel Name

Autosar Select

Calculation Type

Partition Ref Of Channel

▼ CRC Hardware Config

Name

Hardware Instance

Hardware Channel

Dma Channel Enable ☐

Figure 3.15 CRC config Enable "Autosar Select" and "DMA Channel"

- Set "Protocol Type" to CRC_PROTOCOL_16BIT_CCITT_FALSE.

▼ Crc Protocol Info

Name

Protocol Type

Polynomial Value (0x0 -> 0xffffffff)

Write Bit Swap ☐

Read Bit Swap ☐

Inverse Enable (XOR) ☐

Write Byte Swap ☐

Read Byte Swap ☐

Figure 3.16 CRC config protocol is 16BIT_CCITT_FALSE

- Config hardware Channel CRC and DMA

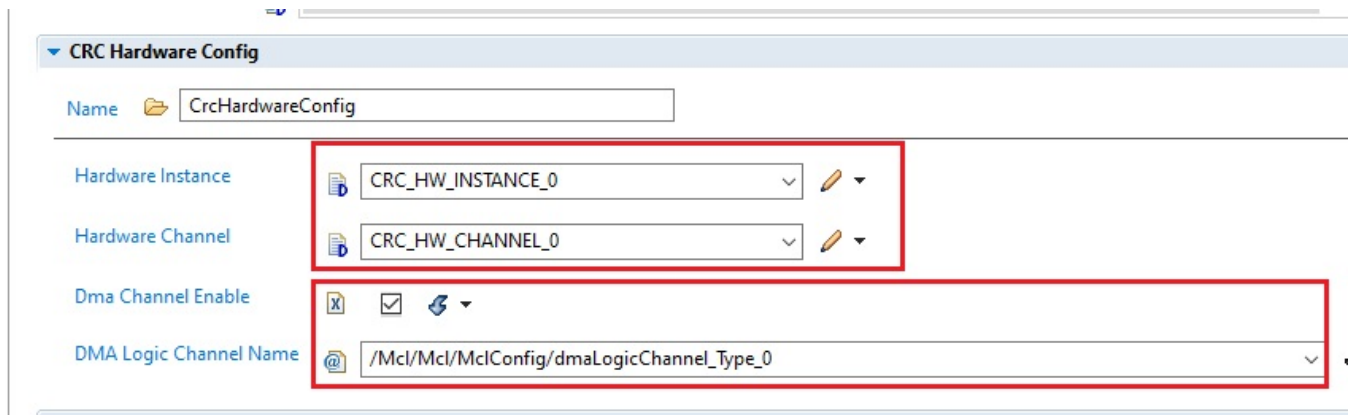


Figure 3.17 Config hardware Channel CRC and DMA

3. Generate the configuration files from Tresos without errors or warnings.
4. Write the application code, following the steps.
 - Call the function `Dma_Ip_Init()` to initialize the DMA driver.
 - Call the function `Crc_Init()` providing it as parameter a NULL pointer.
 - Call the function `Crc_SetChannelCalculate()` to start calculation. Calculation is based on User's logic channel configuration, Crc data path(`pCrcData`), Crc length (`CrcLength`) and start value (`CrcStartValue`). After the calculation is complete, this function will return the result

```

/* Initialize Mcl driver */
Mcl_Init(NULL_PTR);

/* Initialize CRC driver */
Crc_Init(NULL_PTR);

/* Start calculate with CRC_PROTOCOL_16BIT_CCITT_FALSE with SOFTWARE_CALCULATION */
CrcResult = Crc_SetChannelCalculate(CRC_LOGIC_CHANNEL_1, &CRC_data[0], CRC_DATA_SIZE, 0U, TRUE);

```

Figure 3.18 CRC application code

- Note for `Crc_SetChannelCalculate()`: "When DMA is used, the returned result shall be 0U if the DMA Channel did not done transferring all data."

3.6.6.5 Example Calculate CRC with Select Algorithm for table (only support CRC_IP_TABLE_CALCULATION with CRC_PROTOCOL_32BIT_CASTAGNOLI).

To compute CRC checksum use Select Algorithm for table option for CRC_PROTOCOL_32BIT_CASTAGNOLI, please follow the steps below:

1. Step 1 to 3 Please refer "Example Calculate Autosar CRC16 CCITT"
2. In "CRC Channel Configuration", add new CRC logic channel with default config. After that change the Calculate Type to CRC_IP_TABLE_CALCULATION
3. In CRC logic channel configuration, using the following values for the attributes.
 - Set checkbox "Select Algorithm for table calculate" is Enable on Crc General tab.

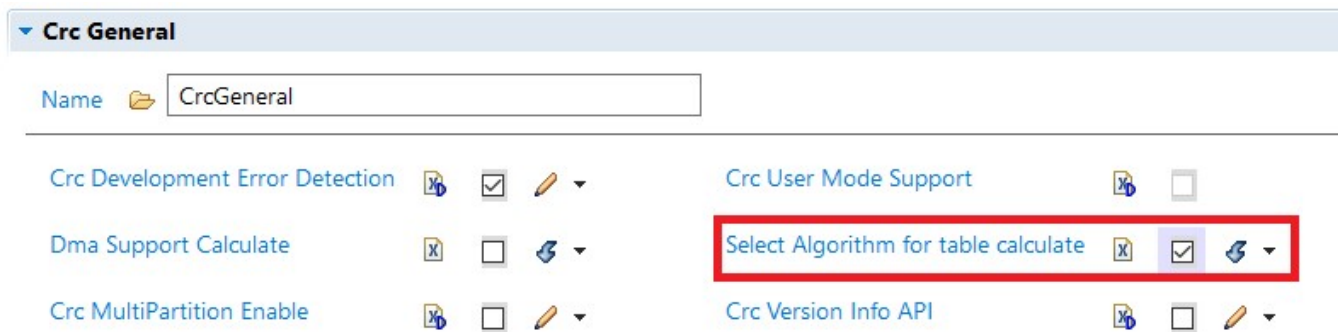


Figure 3.19 CRC Select Algorithm for table calculate Enable

- "Autosar Select" must be NON_AUTOSAR or NON_AUTOSAR_CRC_32CASTAGNOLI, otherwise the "Algorithm Type" could not be selected.
- Choose the supported Protocol and Select the Algorithm Type.

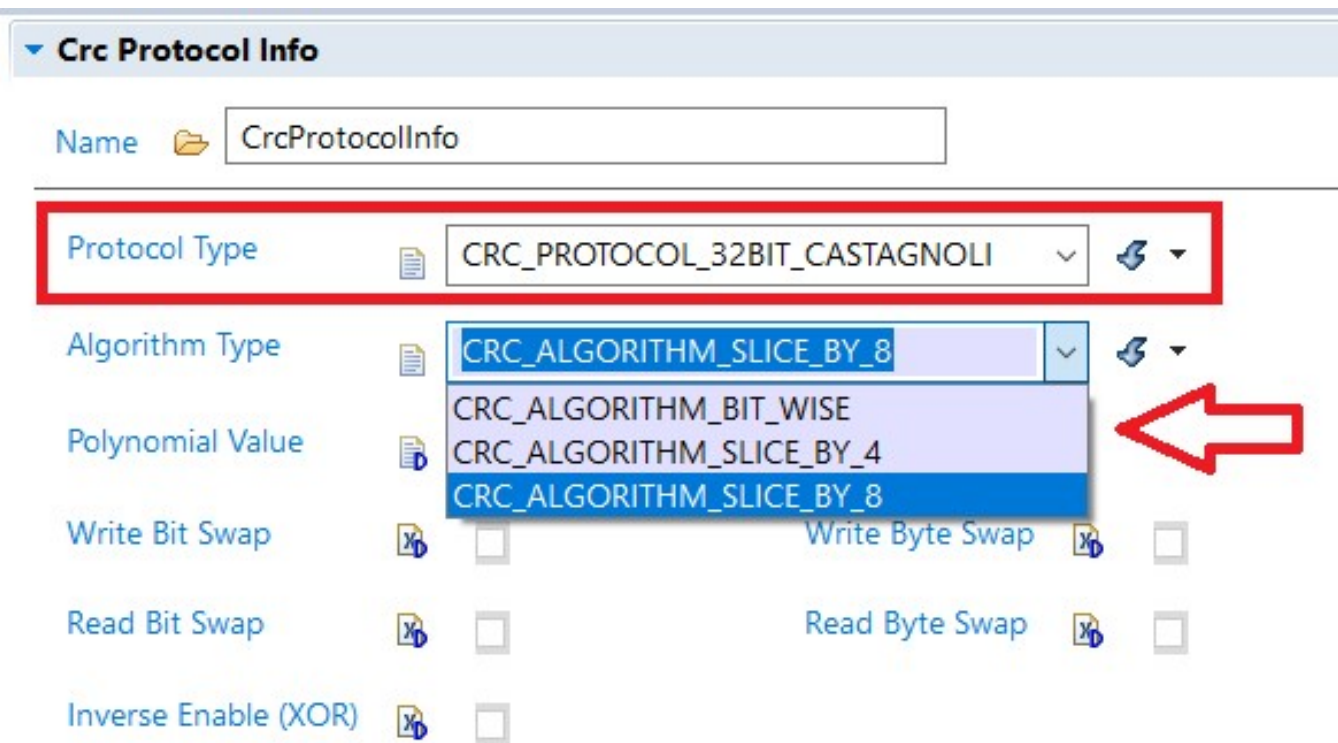


Figure 3.20 CRC select "Algorithm Type"

4. The remaining steps refer examples above.

3.7 Runtime errors

The driver generates the following DET errors at runtime.

Function	Error Code	Condition triggering the error
Crc_Init()	CRC_E_INVALID_POINTER	API is called with a NULL pointer as parameter.
Crc_SetChannelConfig()	CRC_E_INVALID_CHANNEL	API is called with invalid channel ID parameter.
Crc_SetChannelCalculate()	CRC_E_INVALID_CHANNEL	API is called with invalid channel ID parameter.
Crc_SetChannelCalculate()	CRC_E_INVALID_POINTER	API is called with a NULL pointer as parameter.
Crc_GetChannelResult()	CRC_E_INVALID_CHANNEL	API is called with invalid channel ID parameter.
Crc_CalculateCRC8()	CRC_E_INVALID_POINTER	API is called with a NULL pointer as parameter.
Crc_CalculateCRC8()	CRC_E_INVALID_WIDTH_TYPE	API is called with invalid with type support.
Crc_CalculateCRC8H2F()	CRC_E_INVALID_POINTER	API is called with a NULL pointer as parameter.
Crc_CalculateCRC8H2F()	CRC_E_INVALID_WIDTH_TYPE	API is called with invalid with type support.
Crc_CalculateCRC16()	CRC_E_INVALID_POINTER	API is called with a NULL pointer as parameter.
Crc_CalculateCRC16()	CRC_E_INVALID_WIDTH_TYPE	API is called with invalid with type support.
Crc_CalculateCRC16ARC()	CRC_E_INVALID_POINTER	API is called with a NULL pointer as parameter.
Crc_CalculateCRC16ARC()	CRC_E_INVALID_WIDTH_TYPE	API is called with invalid with type support.
Crc_CalculateCRC32()	CRC_E_INVALID_POINTER	API is called with a NULL pointer as parameter.
Crc_CalculateCRC32()	CRC_E_INVALID_WIDTH_TYPE	API is called with invalid with type support.
Crc_CalculateCRC32C()	CRC_E_INVALID_POINTER	API is called with a NULL pointer as parameter.
Crc_CalculateCRC32C()	CRC_E_INVALID_WIDTH_TYPE	API is called with invalid with type support.
Crc_CalculateCRC32P4()	CRC_E_INVALID_POINTER	API is called with a NULL pointer as parameter.
Crc_CalculateCRC32P4()	CRC_E_INVALID_WIDTH_TYPE	API is called with invalid with type support.
Crc_CalculateCRC64()	CRC_E_INVALID_POINTER	API is called with a NULL pointer as parameter.
Crc_CalculateCRC64()	CRC_E_INVALID_WIDTH_TYPE	API is called with invalid with type support.
Crc_GetVersionInfo()	CRC_E_INVALID_POINTER	API is called with a NULL pointer as parameter.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

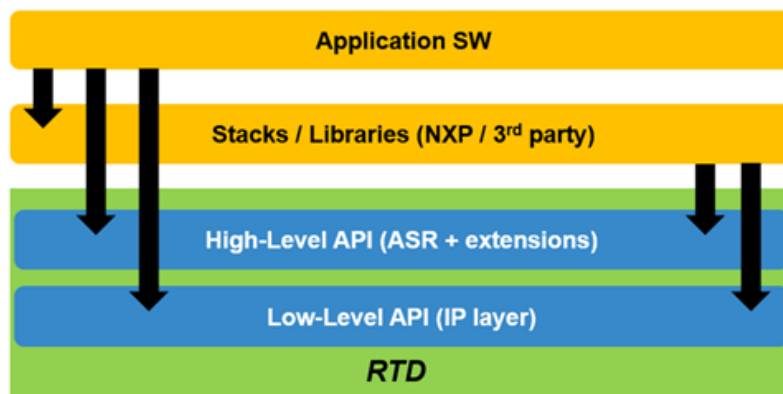


Figure 3.21 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized
 2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
 3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
 4. Initialization of specific drivers
 5. Runtime functionality
 6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity

considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

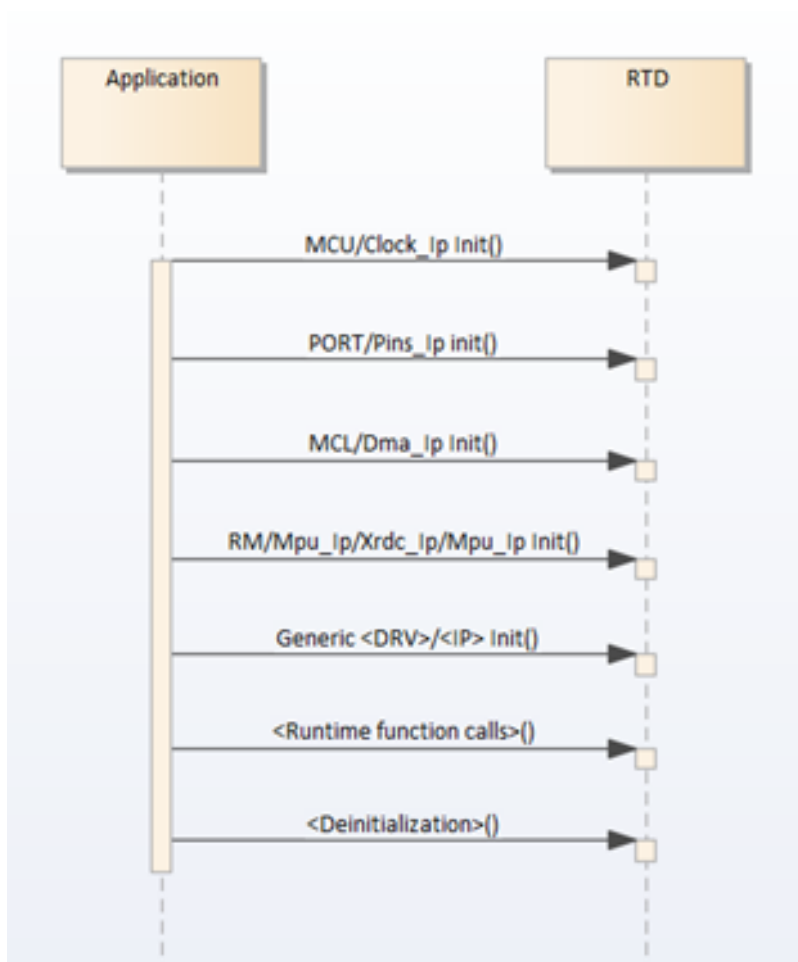


Figure 3.22 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

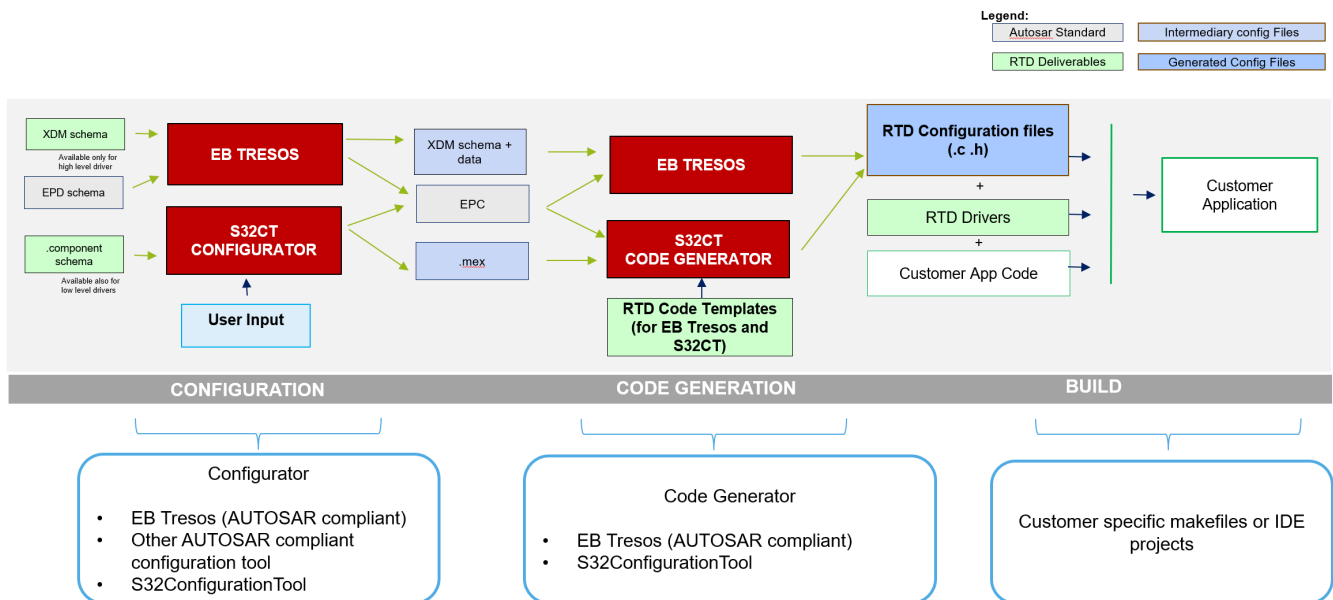


Figure 4.1 Configuration generation flow

- [Crc](#)
- [CRC_IP](#)

4.1 Crc

- Module [Crc](#)
 - Container [CrcGeneral](#)
 - * Parameter [CrcDetectError](#)
 - * Parameter [CrcEnableUserModeSupport](#)
 - * Parameter [CrcDmaSupportEnable](#)
 - * Parameter [CrcSelectAlgorithmSupportEnable](#)
 - * Parameter [CrcMultiPartitionEnable](#)
 - * Parameter [CrcVersionInfoApi](#)
 - * Parameter [Crc8Mode](#)
 - * Parameter [Crc8H2FMode](#)
 - * Parameter [Crc16Mode](#)
 - * Parameter [Crc16ARCMode](#)
 - * Parameter [Crc32Mode](#)
 - * Parameter [Crc32P4Mode](#)
 - * Parameter [Crc32CMode](#)
 - * Parameter [Crc64Mode](#)
 - Container [CrcChannelConfig](#)
 - * Parameter [CrcLogicChannelName](#)
 - * Parameter [CrcAutosarSelect](#)
 - * Parameter [CrcCalculationType](#)
 - * Reference [CrcPartitionRefOfChannel](#)
 - * Container [CrcHardwareConfig](#)
 - Parameter [CrcHwInstance](#)
 - Parameter [CrcHwChannel](#)
 - Parameter [CrcDmaChannelEnable](#)
 - Parameter [CrcDmaTransferSize](#)
 - Parameter [CrcPolynomialSelectionExtendEnable](#)
 - Reference [CrcDmaLogicChannelName](#)
 - * Container [CrcProtocolInfo](#)
 - Parameter [CrcProtocolType](#)
 - Parameter [CrcAlgorithmType](#)
 - Parameter [CrcPolynomialValue](#)
 - Parameter [CrcWriteBitSwap](#)
 - Parameter [CrcWriteByteSwap](#)
 - Parameter [CrcReadBitSwap](#)

- Parameter [CrcReadByteSwap](#)
- Parameter [CrcInversionEnable](#)
- Container [CrcEcucPartitionRefArray](#)
 - * Reference [CrcEcucPartitionRef](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1.1 Module Crc

Vendor specific: Configuration of the Crc (Cyclic Redundancy Check) module.

Included containers:

- [CrcGeneral](#)
- [CrcChannelConfig](#)
- [CrcEcucPartitionRefArray](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	false
supportedConfigVariants	VARIANT-PRE-COMPILE

4.1.2 Container CrcGeneral

Crc General

All general parameters of the Crc driver are collected here.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter CrcDetectError

CRC Development Error Detect

Compile switch to enable/disable development error detection for this module.

Unchecked: Crc Development error detection disabled

Checked : Crc Development error detection enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.4 Parameter CrcEnableUserModeSupport

Crc User Mode Support

When this parameter is enabled, the Crc module will adapt to run from User Mode, with the following measures: Configuring REG_PROT for Crc IPs so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1

For more information and availability on this platform, please see chapter User Mode Support in IM

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.5 Parameter CrcDmaSupportEnable

Dma Support Calculate

Check this in order to be able to use DMA in the Crc driver. Leaving this unchecked will allow the Crc driver to compile with no dependencies from the Mcl driver.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.6 Parameter CrcSelectAlgorithmSupportEnable

Select Algorithm Support Calculate

Check this in order to be able to select algorithm for the Crc driver when calculate CRC on table mode. Leaving this unchecked will allow the Crc driver using default algorithm (bitwise) for calculating CRC on table mode.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.7 Parameter CrcMultiPartitionEnable

Crc MultiPartition Enable

This parameter globally enables the possibility to support MultiPartition. If this parameter is enabled, at least one EcucPartition needs to be defined (in all variants).

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.8 Parameter CrcVersionInfoApi

CRC VersionInfo Api

Compile switch to enable/disable the version information API.

Checked : API enabled

Unchecked: API disabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.9 Parameter Crc8Mode

Switch to select one of the available Crc 8-bit (SAE J1850) calculation methods

Note: If large data blocks have to be calculated (>32 bytes, depending on performance of processor platform), the table based calculation or hardware method should be configured for the function Crc_CalculateCRC8 in order to decrease the calculation time.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	False
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false

Property	Value
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_8_TABLE
literals	['CRC_8_TABLE', 'CRC_8_HARDWARE', 'CRC_8_RUNTIME']

4.1.10 Parameter Crc8H2FMode

Switch to select one of the available Crc 8-bit (2Fh polynomial) calculation methods

Note: If large data blocks have to be calculated (>32 bytes, depending on performance of processor platform), the table based calculation or hardware method should be configured for the function Crc_CalculateCRC8H2F in order to decrease the calculation time.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	False
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_8H2F_TABLE
literals	['CRC_8H2F_TABLE', 'CRC_8H2F_HARDWARE', 'CRC_8H2F_RUNTIME']

4.1.11 Parameter Crc16Mode

Switch to select one of the available Crc 16-bit (CCITT-FALSE) calculation methods

Note: If large data blocks have to be calculated (>32 bytes, depending on performance of processor platform), the table based calculation or hardware method should be configured for the function Crc_CalculateCRC16 in order to decrease the calculation time.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	False
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_16_TABLE
literals	['CRC_16_TABLE', 'CRC_16_HARDWARE', 'CRC_16_RUNTIME']

4.1.12 Parameter Crc16ARCMode

Switch to select one of the available CRC-16/ARC (polynomial 8005) calculation methods

Note: If large data blocks have to be calculated (>32 bytes, depending on performance of processor platform), the table based calculation method should be configured for the function `Crc_CalculateCRC16ARC` in order to decrease the calculation time.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	False
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_16ARC_TABLE
literals	['CRC_16ARC_TABLE', 'CRC_16ARC_HARDWARE', 'CRC_16ARC_RUNTIME']

4.1.13 Parameter Crc32Mode

Switch to select one of the available Crc 32-bit (IEEE-802.3 CRC32 Ethernet Standard) calculation methods

Note: If large data blocks have to be calculated (>32 bytes, depending on performance of processor platform), the table based calculation or hardware method should be configured for the function `Crc_CalculateCRC32` in order to decrease the calculation time.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	False
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_32_TABLE
literals	['CRC_32_TABLE', 'CRC_32_HARDWARE', 'CRC_32_RUNTIME']

4.1.14 Parameter Crc32P4Mode

Switch to select one of the available Crc 32-bit E2E Profile 4 calculation methods.

Note: If large data blocks have to be calculated (>32 bytes, depending on performance of processor platform), the table based calculation method should be configured for the function `Crc_CalculateCRC32P4` in order to decrease the calculation time.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	False
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_32P4_TABLE
literals	['CRC_32P4_TABLE', 'CRC_32P4_HARDWARE', 'CRC_32P4_RUNTIME']

4.1.15 Parameter Crc32CMode

Switch to select one of the available Crc 32-bit Castagnoli calculation methods.

Note: If large data blocks have to be calculated (>32 bytes, depending on performance of processor platform), the table based calculation method should be configured for the function Crc_CalculateCRC32C in order to decrease the calculation time.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_32C_TABLE
literals	['CRC_32C_TABLE', 'CRC_32C_HARDWARE', 'CRC_32C_RUNTIME']

4.1.16 Parameter Crc64Mode

Switch to select one of the available Crc 64-bit calculation methods

Note: If large data blocks have to be calculated (>64 bytes, depending on performance of processor platform), the table based calculation method should be configured for the function Crc_CalculateCRC64 in order to decrease the calculation time.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	False
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false

Configuration Parameters

Property	Value
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_64_TABLE
literals	['CRC_64_TABLE', 'CRC_64_HARDWARE', 'CRC_64_RUNTIME']

4.1.17 Container CrcChannelConfig

Crc Channels Configuration

Configuration of an individual Crc channel. Symbolic names will be generated for each channel.

Included subcontainers:

- [CrcHardwareConfig](#)
- [CrcProtocolInfo](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.18 Parameter CrcLogicChannelName

Logic Channel Name

Channel used for Crc calculation

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_LOGIC_CHANNEL_0

4.1.19 Parameter CrcAutosarSelect

Autosar Selection

Select the Autosar Mode to run.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NON_AUTOSAR
literals	['NON_AUTOSAR', 'NON_AUTOSAR_CRC_32CASTAGNOLI', 'AUTOSAR_CRC_8', 'AUTOSAR_CRC_8H2F', 'AUTOSAR_CRC_16', 'AUTOSAR_CRC_16ARC', 'AUTOSAR_CRC_32', 'AUTOSAR_CRC_32P4', 'AUTOSAR_CRC_64']

4.1.20 Parameter CrcCalculationType

Calculation Type

Select Crc Calculation Type.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_IP_TABLE_CALCULATION
literals	['CRC_IP_TABLE_CALCULATION', 'CRC_IP_HARDWARE_CALCULATION', 'CRC_IP_RUNTIME_CALCULATION']

4.1.21 Reference CrcPartitionRefOfChannel

Partition Ref Of Channel

Maps a Crc hardware unit to zero or one ECUC partition to limit the access to this hardware unit. The ECUC partitions referenced are a subset of the ECUC partitions where the Crc driver is mapped to.

Tags: atp.Status=draft

Property	Value
type	ECUC-REFERENCE-DEF

Property	Value
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.22 Container CrcHardwareConfig

This container contains the hardware configuration parameters of the Crc module.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.23 Parameter CrcHwInstance

Hardware Instance

Identifies the Crc Hardware Instance.

Note:

- Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	CRC_HW_INSTANCE_0
literals	['CRC_HW_INSTANCE_0']

4.1.24 Parameter CrcHwChannel

Hardware Channel

Selects one of the Crc hardware channels available on the device.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	CRC_HW_CHANNEL_0
literals	['CRC_HW_CHANNEL_0']

4.1.25 Parameter CrcDmaChannelEnable

Dma Channel Enable

Checked : Enabled

Unchecked: Disabled

Note: DMA mode does not support for CRC Autosar Library

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.26 Parameter CrcDmaTransferSize

Switch to select one of the available DMA Transfer Sizes

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	False

Configuration Parameters

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_DMA_TRANSFER_SIZE_1_BYTE
literals	['CRC_DMA_TRANSFER_SIZE_1_BYTE', 'CRC_DMA_TRANSFER_SIZE_2_BYTE', 'CRC_DMA_TRANSFER_SIZE_4_BYTE']

4.1.27 Parameter CrcPolynomialSelectionExtendEnable

Polynomial selection extend function

Check this in order to be able to use Polynomial selection extend function in the Crc driver (POLYG_EXT bit field).

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.28 Reference CrcDmaLogicChannelName

DMA Logic Channel Name

DMA Logic Channel is used for this Crc logic channel.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False

Property	Value
destination	/TS_T40D2M30I0R0/Mcl/MclConfig/dmaLogicChannel_Type

4.1.29 Container CrcProtocolInfo

This container configuration parameters of protocol type

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.30 Parameter CrcProtocolType

Protocol Type

Identifies the Crc Protocol Type.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_PROTOCOL_8BIT_SAE_J1850
literals	['CRC_PROTOCOL_8BIT_SAE_J1850', 'CRC_PROTOCOL_8BIT_H2F', 'CRC_PROTOCOL_16BIT_CCITT_FALSE', 'CRC_PROTOCOL_16BIT_ARC', 'CRC_PROTOCOL_32BIT_ETHERNET', 'CRC_PROTOCOL_32BIT_E2E_P4', 'CRC_PROTOCOL_32BIT_CASTAGNOLI', 'CRC_PROTOCOL_64BIT_ECMA', 'CRC_PROTOCOL_8BIT_CUSTOM', 'CRC_PROTOCOL_16BIT_CUSTOM', 'CRC_PROTOCOL_32BIT_CUSTOM', 'CRC_PROTOCOL_64BIT_CUSTOM']

4.1.31 Parameter CrcAlgorithmType

Algorithm Type

Configuration Parameters

Identifies the Crc Algorithm Type.

Note: Implementation Specific Parameter.

Only CRC_PROTOCOL_32BIT_CASTAGNOLI support the slice-by-N algorithm type.

When using slice-by-N algorithm the length of data needed to be calculate CRC must be a multiple of N.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CRC_ALGORITHM_BIT_WISE
literals	['CRC_ALGORITHM_BIT_WISE', 'CRC_ALGORITHM_SLICE_BY_4', 'CRC_ALGORITHM_SLICE_BY_8']

4.1.32 Parameter CrcPolynomialValue

Polynomial Value

Start value when the algorithm starts in process Calculate CRC

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0x

4.1.33 Parameter CrcWriteBitSwap

Write Bit Swap

Enumerator that defines Crc Write data bitwise functionality.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.34 Parameter CrcWriteByteSwap

Write Bit Swap

Enumerator that defines Crc Write data bitwise functionality.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.35 Parameter CrcReadBitSwap

Read Bit Swap

Enumerator that defines Crc Read data bitwise functionality.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.36 Parameter CrcReadByteSwap

Read Byte Swap

Enumerator that defines Crc Read data bitwise functionality.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.37 Parameter CrcInversionEnable

Inverse Enable (XOR)

The result shall be complement(inversion) of the actual checksum.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.38 Container CrcEcucPartitionRefArray

Crc Ecuc Partition Ref Array.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false

Property	Value
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.39 Reference CrcEcucPartitionRef

Maps the Crc driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.40 Container CommonPublishedInformation

Common Published Information

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.41 Parameter ArReleaseMajorVersion

AUTOSAR Release Major Version

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Configuration Parameters

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.42 Parameter ArReleaseMinorVersion

AUTOSAR Release Minor Version

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.43 Parameter ArReleaseRevisionVersion

AUTOSAR Release Revision Version

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0

Property	Value
min	0

4.1.44 Parameter ModuleId

Module ID

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	201
max	201
min	201

4.1.45 Parameter SwMajorVersion

Software Major Version

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.46 Parameter SwMinorVersion

Software Minor Version

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Configuration Parameters

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.47 Parameter SwPatchVersion

Software Patch Version

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.48 Parameter VendorApiInfix

Vendor Api Infix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" an api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.1.49 Parameter VendorId

Vendor Id

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.2 CRC_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Crc_Ip](#)
 - Container [CrcGeneral](#)
 - * Parameter [CrcDetectError](#)
 - * Parameter [CrcEnableUserModeSupport](#)
 - * Parameter [CrcDmaSupportEnable](#)
 - * Parameter [CrcSelectAlgorithmSupportEnable](#)
 - Container [CrcChannelConfig](#)

4.2.1 Module Crc_Ip

Crc_Ip Configuration

Included containers:

- [CrcGeneral](#)
- [CrcChannelConfig](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	false
supportedConfigVariants	VARIANT-PRE-COMPILE



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

CRC HLD Driver	48
CRC IPL Driver	62

Chapter 6

Module Documentation

6.1 CRC HLD Driver

6.1.1 Detailed Description

Data Structures

- struct [Crc_PartitionType](#)
CRC Partition Configuration Type. [More...](#)
- struct [Crc_InitType](#)
Initialization data for the CRC driver. [More...](#)

Macros

- `#define CDD_CRC_MODULE_ID`
Parameters that shall be published within the Crc driver header file and also in the module's description file.
- `#define CRC_INSTANCE_ID`
ID of CRC Instance.
- `#define CRC_CALCULATECRC8_ID`
API service ID for Crc_CalculateCRC8 function.
- `#define CRC_CALCULATECRC16_ID`
API service ID for Crc_CalculateCRC16 function.
- `#define CRC_CALCULATECRC32_ID`
API service ID for Crc_CalculateCRC32 function.
- `#define CRC_GETVERSIONINFO_ID`
API service ID for Crc_GetVersionInfo function.
- `#define CRC_CALCULATECRC8H2F_ID`
API service ID for Crc_CalculateCRC8H2F function.
- `#define CRC_CALCULATECRC32P4_ID`
API service ID for Crc_CalculateCRC32P4 function.
- `#define CRC_CALCULATECRC64_ID`
API service ID for Crc_CalculateCRC64 function.
- `#define CRC_CALCULATECRC16ARC_ID`

- API service ID for Crc_CalculateCRC16ARC function.*

 - #define [CRC_INIT_ID](#)

API service ID for Crc_Init function.
- #define [CRC_SETCHANNELCONFIG_ID](#)

API service ID for Crc_SetChannelConfig function.
- #define [CRC_SETCHANNELCALCULATE_ID](#)

API service ID for Crc_SetChannelCalculate function.
- #define [CRC_GETCHANNELRESULT_ID](#)

API service ID for Crc_GetChannelResult function.
- #define [CRC_CALCULATECRC32C_ID](#)

API service ID for Crc_CalculateCRC32C function.
- #define [CRC_E_INVALID_CHANNEL](#)

API service is called with wrong channel identifier.
- #define [CRC_E_INVALID_POINTER](#)

API service is called with NULL pointer parameter.
- #define [CRC_E_PARAM_CONFIG](#)

The CRC module is not properly configured.
- #define [CRC_E_INIT_FAILED](#)

The CRC module is not properly initialized.
- #define [CRC_TYPES_VENDOR_ID](#)

Parameters that shall be published within the Crc driver header file and also in the module's description file.

Types Reference

- typedef [Crc_Ip_LogicChannelConfigType](#) [Crc_ChannelConfigType](#)

This type contains the CRC Channel Configuration.
- typedef [Crc_Ip_ProtocolType](#) [Crc_ProtocolType](#)

This type contains the CRC Channel Protocol.

Function Reference

- void [Crc_Init](#) (const [Crc_InitType](#) *ConfigPtr)

This service will store the Crc driver installation configuration based on user configuration.
- void [Crc_SetChannelConfig](#) (const uint32 Channel, const [Crc_ChannelConfigType](#) *pxChannelConfig)

This function initializes the CRC Channel configuration by logicChannel's info.
- uint64 [Crc_SetChannelCalculate](#) (const uint32 Channel, const uint8 *pCrcData, const uint32 CrcLength, const uint64 CrcStartValue, const boolean IsFirstCall)

This function shall start algorithm calculate CRC.
- uint64 [Crc_GetChannelResult](#) (const uint32 Channel)

This function get result CRC after finish calculated.
- uint8 [Crc_CalculateCRC8](#) (const uint8 *Crc_DataPtr, uint32 Crc_Length, uint8 Crc_StartValue8, boolean Crc_IsFirstCall)

This function calculates CRC8 and returns the result.

- uint8 [Crc_CalculateCRC8H2F](#) (const uint8 *Crc_DataPtr, uint32 Crc_Length, uint8 Crc_StartValue8H2F, boolean Crc_IsFirstCall)

This function calculates CRC8H2F and returns the result.

- uint16 [Crc_CalculateCRC16](#) (const uint8 *Crc_DataPtr, uint32 Crc_Length, uint16 Crc_StartValue16, boolean Crc_IsFirstCall)

This function calculates CRC16 and returns the result.

- uint16 [Crc_CalculateCRC16ARC](#) (const uint8 *Crc_DataPtr, uint32 Crc_Length, uint16 Crc_StartValue16, boolean Crc_IsFirstCall)

This function calculates CRC16 and returns the result.

- uint32 [Crc_CalculateCRC32](#) (const uint8 *Crc_DataPtr, uint32 Crc_Length, uint32 Crc_StartValue32, boolean Crc_IsFirstCall)

This function calculates CRC32 and returns the result.

- uint32 [Crc_CalculateCRC32P4](#) (const uint8 *Crc_DataPtr, uint32 Crc_Length, uint32 Crc_StartValue32, boolean Crc_IsFirstCall)

This function calculates CRC32P4 and returns the result.

- uint64 [Crc_CalculateCRC64](#) (const uint8 *Crc_DataPtr, uint32 Crc_Length, uint64 Crc_StartValue64, boolean Crc_IsFirstCall)

This function calculates CRC64 and returns the result.

- uint32 [Crc_CalculateCRC32C](#) (const uint8 *Crc_DataPtr, uint32 Crc_Length, uint32 Crc_StartValue32, boolean Crc_IsFirstCall)

This function calculates CRC32C and returns the result.

- void [Crc_GetVersionInfo](#) (Std_VersionInfoType *Versioninfo)

Software module version query.

6.1.2 Data Structure Documentation

6.1.2.1 struct Crc_PartitionType

CRC Partition Configuration Type.

The Channel is identified by the following Partition

Definition at line 111 of file Crc_Types.h.

6.1.2.2 struct Crc_InitType

Initialization data for the CRC driver.

A pointer to such a structure is provided to the CRC initialization routines for configuration.

Definition at line 123 of file Crc_Types.h.

6.1.3 Macro Definition Documentation

6.1.3.1 CDD_CRC_MODULE_ID

```
#define CDD_CRC_MODULE_ID
```

Parameters that shall be published within the Crc driver header file and also in the module's description file.

Definition at line 61 of file CDD_Crc.h.

6.1.3.2 CRC_INSTANCE_ID

```
#define CRC_INSTANCE_ID
```

ID of CRC Instance.

Parameters used when raising an error/exception

Definition at line 131 of file CDD_Crc.h.

6.1.3.3 CRC_CALCULATECRC8_ID

```
#define CRC_CALCULATECRC8_ID
```

API service ID for Crc_CalculateCRC8 function.

Parameters used when raising an error/exception

Definition at line 142 of file CDD_Crc.h.

6.1.3.4 CRC_CALCULATECRC16_ID

```
#define CRC_CALCULATECRC16_ID
```

API service ID for Crc_CalculateCRC16 function.

Parameters used when raising an error/exception

Definition at line 148 of file CDD_Crc.h.

6.1.3.5 CRC_CALCULATECRC32_ID

```
#define CRC_CALCULATECRC32_ID
```

API service ID for Crc_CalculateCRC32 function.

Parameters used when raising an error/exception

Definition at line 154 of file CDD_Crc.h.

6.1.3.6 CRC_GETVERSIONINFO_ID

```
#define CRC_GETVERSIONINFO_ID
```

API service ID for Crc_GetVersionInfo function.

Parameters used when raising an error/exception

Definition at line 160 of file CDD_Crc.h.

6.1.3.7 CRC_CALCULATECRC8H2F_ID

```
#define CRC_CALCULATECRC8H2F_ID
```

API service ID for Crc_CalculateCRC8H2F function.

Parameters used when raising an error/exception

Definition at line 166 of file CDD_Crc.h.

6.1.3.8 CRC_CALCULATECRC32P4_ID

```
#define CRC_CALCULATECRC32P4_ID
```

API service ID for Crc_CalculateCRC32P4 function.

Parameters used when raising an error/exception

Definition at line 172 of file CDD_Crc.h.

6.1.3.9 CRC_CALCULATECRC64_ID

```
#define CRC_CALCULATECRC64_ID
```

API service ID for Crc_CalculateCRC64 function.

Parameters used when raising an error/exception

Definition at line 178 of file CDD_Crc.h.

6.1.3.10 CRC_CALCULATECRC16ARC_ID

```
#define CRC_CALCULATECRC16ARC_ID
```

API service ID for Crc_CalculateCRC16ARC function.

Parameters used when raising an error/exception

Definition at line 184 of file CDD_Crc.h.

6.1.3.11 CRC_INIT_ID

```
#define CRC_INIT_ID
```

API service ID for Crc_Init function.

Parameters used when raising an error/exception

Definition at line 195 of file CDD_Crc.h.

6.1.3.12 CRC_SETCHANNELCONFIG_ID

```
#define CRC_SETCHANNELCONFIG_ID
```

API service ID for Crc_SetChannelConfig function.

Parameters used when raising an error/exception

Definition at line 201 of file CDD_Crc.h.

6.1.3.13 CRC_SETCHANNELCALCULATE_ID

```
#define CRC_SETCHANNELCALCULATE_ID
```

API service ID for Crc_SetChannelCalculate function.

Parameters used when raising an error/exception

Definition at line 207 of file CDD_Crc.h.

6.1.3.14 CRC_GETCHANNELRESULT_ID

```
#define CRC_GETCHANNELRESULT_ID
```

API service ID for Crc_GetChannelResult function.

Parameters used when raising an error/exception

Definition at line 213 of file CDD_Crc.h.

6.1.3.15 CRC_CALCULATECRC32C_ID

```
#define CRC_CALCULATECRC32C_ID
```

API service ID for Crc_CalculateCRC32C function.

Parameters used when raising an error/exception

Definition at line 219 of file CDD_Crc.h.

6.1.3.16 CRC_E_INVALID_CHANNEL

```
#define CRC_E_INVALID_CHANNEL
```

API service is called with wrong channel identifier.

Parameter is used when raising a Det error

Definition at line 230 of file CDD_Crc.h.

6.1.3.17 CRC_E_INVALID_POINTER

```
#define CRC_E_INVALID_POINTER
```

API service is called with NULL pointer parameter.

Parameter is used when raising a Det error

Definition at line 237 of file CDD_Crc.h.

6.1.3.18 CRC_E_PARAM_CONFIG

```
#define CRC_E_PARAM_CONFIG
```

The CRC module is not properly configured.

Parameter is used when raising a Det error

Definition at line 244 of file CDD_Crc.h.

6.1.3.19 CRC_E_INIT_FAILED

```
#define CRC_E_INIT_FAILED
```

The CRC module is not properly initialized.

Parameter is used when raising a Det error

Definition at line 251 of file CDD_Crc.h.

6.1.3.20 CRC_TYPES_VENDOR_ID

```
#define CRC_TYPES_VENDOR_ID
```

Parameters that shall be published within the Crc driver header file and also in the module's description file.

Definition at line 60 of file Crc_Types.h.

6.1.4 Types Reference

6.1.4.1 Crc_ChannelConfigType

```
typedef Crc_Ip_LogicChannelConfigType Crc_ChannelConfigType
```

This type contains the CRC Channel Configuration.

The Channel is identified by the following structure configure parameter for each CRC Channel

Definition at line 266 of file CDD_Crc.h.

6.1.4.2 Crc_ProtocolType

```
typedef Crc_Ip_ProtocolType Crc_ProtocolType
```

This type contains the CRC Channel Protocol.

The Protocol shall be selected from the available list. Software CRC supports all protocols. Table based CRC support the following protocols: Hardware CRC supports implementation specific protocols.

Definition at line 136 of file Crc_Types.h.

6.1.5 Function Reference

6.1.5.1 Crc_Init()

```
void Crc_Init (
    const Crc_InitType * ConfigPtr )
```

This service will store the Crc driver installation configuration based on user configuration.

This service is a non-reentrant function that shall store user configuration. The initialization is applied for the enabled IPs, configured statically.

Parameters

in	<i>pxCrcInit</i>	- Pointer to the Crc specific configuration structure that contains static configuration.
----	------------------	---

Returns

void

6.1.5.2 Crc_SetChannelConfig()

```
void Crc_SetChannelConfig (
    const uint32 Channel,
    const Crc_ChannelConfigType * pxChannelConfig )
```

This function initializes the CRC Channel configuration by logicChannel's info.

This service is a reentrant function that shall initialize parameters list for the CRC Channel. The list is composed of an array of Crc Channel parameters settings.

Parameters

in	<i>Channel</i>	- Logic Channel Tag defined by the user.
in	<i>pxChannelConfig</i>	- Pointer to the Logic Channel Config

Returns

void

6.1.5.3 Crc_SetChannelCalculate()

```
uint64 Crc_SetChannelCalculate (
```

```
const uint32 Channel,
const uint8 * pCrcData,
const uint32 CrcLength,
const uint64 CrcStartValue,
const boolean IsFirstCall )
```

This function shall start algorithm calculate CRC.

This service is a reentrant function that shall start the Channel to calculate the CRC using the configured algorithm.

Parameters

in	<i>Channel</i>	- Logic Channel Tag defined by the user.
in	<i>*pCrcData</i>	- Pointer to the Crc Data Input.
in	<i>CrcLength</i>	- Length of crcDataPtr block to be calculated in bytes
in	<i>CrcStartValue</i>	- Start value (seed Value) when the algorithm starts.
in	<i>IsFirstCall</i>	- TRUE: ignore CrcStartValue the initialization value is known by the chosen algorithm. • FALSE: initialization CrcStartValue is interpreted to be the return value of the previous function

Returns

- 32 bit result of CRC calculation

6.1.5.4 Crc_GetChannelResult()

```
uint64 Crc_GetChannelResult (
    const uint32 Channel )
```

This function get result CRC after finish calculated.

This service is a function that shall get result CRC calculated by CRC Channel Allocated

Parameters

in	<i>Channel</i>	- Logic Channel Tag defined by the user.
----	----------------	--

Returns

Result of CRC calculation.

6.1.5.5 Crc_CalculateCRC8()

```
uint8 Crc_CalculateCRC8 (
    const uint8 * Crc_DataPtr,
    uint32 Crc_Length,
    uint8 Crc_StartValue8,
    boolean Crc_IsFirstCall )
```

This function calculates CRC8 and returns the result.

This service is a reentrant function that shall perform a CRC8 calculation on Crc_Length data bytes, pointed to by Crc_DataPtr, with the starting value of Crc_StartValue8

Parameters

in	<i>*Crc_DataPtr</i>	- Pointer to start address of data block to be calculated.
in	<i>Crc_Length</i>	- Length of data block to be calculated in bytes.
in	<i>Crc_StartValue8</i>	- Length value of the DataPtr.
in	<i>Crc_IsFirstCall</i>	- TRUE: First call in a sequence or individual CRC calculation. Start from initial value, ignore Crc_StartValue8. FALSE: Subsequent call in a call sequence. Crc_StartValue8 is interpreted to be the return value of the previous function call.

Returns

8 bit result of CRC calculation.

6.1.5.6 Crc_CalculateCRC8H2F()

```
uint8 Crc_CalculateCRC8H2F (
    const uint8 * Crc_DataPtr,
    uint32 Crc_Length,
    uint8 Crc_StartValue8H2F,
    boolean Crc_IsFirstCall )
```

This function calculates CRC8H2F and returns the result.

This service is a reentrant function that shall perform a CRC8H2F calculation on Crc_Length data bytes, pointed to by Crc_DataPtr, with the starting value of Crc_StartValue8H2F

Parameters

in	<i>*Crc_DataPtr</i>	- Pointer to start address of data block to be calculated.
in	<i>Crc_Length</i>	- Length of data block to be calculated in bytes.
in	<i>Crc_StartValue8H2F</i>	- Length value of the DataPtr.
in	<i>Crc_IsFirstCall</i>	- TRUE: First call in a sequence or individual CRC calculation. Start from initial value, ignore Crc_StartValue8H2F. FALSE: Subsequent call in a call sequence. Crc_StartValue8H2F is interpreted to be the return value of the previous function call.

Returns

8 bit result of CRC calculation.

6.1.5.7 Crc_CalculateCRC16()

```
uint16 Crc_CalculateCRC16 (
    const uint8 * Crc_DataPtr,
    uint32 Crc_Length,
    uint16 Crc_StartValue16,
    boolean Crc_IsFirstCall )
```

This function calculates CRC16 and returns the result.

Module Documentation

This service is a reentrant function that shall perform a CRC16 calculation on `Crc_Length` data bytes, pointed to by `Crc_DataPtr`, with the starting value of `Crc_StartValue16`

Parameters

in	<i>*Crc_DataPtr</i>	- Pointer to start address of data block to be calculated.
in	<i>Crc_Length</i>	- Length of data block to be calculated in bytes.
in	<i>Crc_StartValue16</i>	- Length value of the DataPtr.
in	<i>Crc_IsFirstCall</i>	- TRUE: First call in a sequence or individual CRC calculation. Start from initial value, ignore <code>Crc_StartValue16</code> . FALSE: Subsequent call in a call sequence. <code>Crc_StartValue16</code> is interpreted to be the return value of the previous function call.

Returns

16 bit result of CRC calculation.

6.1.5.8 Crc_CalculateCRC16ARC()

```
uint16 Crc_CalculateCRC16ARC (  
    const uint8 * Crc_DataPtr,  
    uint32 Crc_Length,  
    uint16 Crc_StartValue16,  
    boolean Crc_IsFirstCall )
```

This function calculates CRC16 and returns the result.

This service is a reentrant function that shall perform a CRC16 calculation on `Crc_Length` data bytes, pointed to by `Crc_DataPtr`, with the starting value of `Crc_StartValue16`

Parameters

in	<i>*Crc_DataPtr</i>	- Pointer to start address of data block to be calculated.
in	<i>Crc_Length</i>	- Length of data block to be calculated in bytes.
in	<i>Crc_StartValue16</i>	- Length value of the DataPtr.
in	<i>Crc_IsFirstCall</i>	- TRUE: First call in a sequence or individual CRC calculation. Start from initial value, ignore <code>Crc_StartValue16</code> . FALSE: Subsequent call in a call sequence. <code>Crc_StartValue16</code> is interpreted to be the return value of the previous function call.

Returns

16 bit result of CRC calculation.

6.1.5.9 Crc_CalculateCRC32()

```
uint32 Crc_CalculateCRC32 (  
    const uint8 * Crc_DataPtr,  
    uint32 Crc_Length,
```

```
uint32 Crc_StartValue32,
boolean Crc_IsFirstCall )
```

This function calculates CRC32 and returns the result.

This service is a reentrant function that shall perform a CRC32 calculation on Crc_Length data bytes, pointed to by Crc_DataPtr, with the starting value of Crc_StartValue32

Parameters

in	<i>*Crc_DataPtr</i>	- Pointer to start address of data block to be calculated.
in	<i>Crc_Length</i>	- Length of data block to be calculated in bytes.
in	<i>Crc_StartValue32</i>	- Length value of the DataPtr.
in	<i>Crc_IsFirstCall</i>	- TRUE: First call in a sequence or individual CRC calculation. Start from initial value, ignore Crc_StartValue32. FALSE: Subsequent call in a call sequence. Crc_StartValue32 is interpreted to be the return value of the previous function call.

Returns

32 bit result of CRC calculation.

6.1.5.10 Crc_CalculateCRC32P4()

```
uint32 Crc_CalculateCRC32P4 (
    const uint8 * Crc_DataPtr,
    uint32 Crc_Length,
    uint32 Crc_StartValue32,
    boolean Crc_IsFirstCall )
```

This function calculates CRC32P4 and returns the result.

This service is a reentrant function that shall perform a CRC32P4 calculation on Crc_Length data bytes, pointed to by Crc_DataPtr, with the starting value of Crc_StartValue32

Parameters

in	<i>*Crc_DataPtr</i>	- Pointer to start address of data block to be calculated.
in	<i>Crc_Length</i>	- Length of data block to be calculated in bytes.
in	<i>Crc_StartValue32</i>	- Length value of the DataPtr.
in	<i>Crc_IsFirstCall</i>	- TRUE: First call in a sequence or individual CRC calculation. Start from initial value, ignore Crc_StartValue32. FALSE: Subsequent call in a call sequence. Crc_StartValue32 is interpreted to be the return value of the previous function call.

Returns

32 bit result of CRC calculation.

6.1.5.11 Crc_CalculateCRC64()

```
uint64 Crc_CalculateCRC64 (
    const uint8 * Crc_DataPtr,
    uint32 Crc_Length,
    uint64 Crc_StartValue64,
    boolean Crc_IsFirstCall )
```

This function calculates CRC64 and returns the result.

This service is a reentrant function that shall perform a CRC64 calculation on Crc_Length data bytes, pointed to by Crc_DataPtr, with the starting value of Crc_StartValue64

Parameters

in	<i>*Crc_DataPtr</i>	- Pointer to start address of data block to be calculated.
in	<i>Crc_Length</i>	- Length of data block to be calculated in bytes.
in	<i>u64Crc_StartValue</i>	- Length value of the DataPtr.
in	<i>Crc_IsFirstCall</i>	- TRUE: First call in a sequence or individual CRC calculation. Start from initial value, ignore Crc_StartValue64. • FALSE: Subsequent call in a call sequence. Crc_StartValue64 is interpreted to be the return value of the previous function call.

Returns

64 bit result of CRC calculation.

6.1.5.12 Crc_CalculateCRC32C()

```
uint32 Crc_CalculateCRC32C (
    const uint8 * Crc_DataPtr,
    uint32 Crc_Length,
    uint32 Crc_StartValue32,
    boolean Crc_IsFirstCall )
```

This function calculates CRC32C and returns the result.

This service is a reentrant function that shall perform a CRC32C calculation on Crc_Length data bytes, pointed to by Crc_DataPtr, with the starting value of Crc_StartValue32

Parameters

in	<i>*Crc_DataPtr</i>	- Pointer to start address of data block to be calculated.
in	<i>Crc_Length</i>	- Length of data block to be calculated in bytes.
in	<i>Crc_StartValue32</i>	- Length value of the DataPtr.

Parameters

in	<i>Crc_IsFirstCall</i>	- TRUE: First call in a sequence or individual CRC calculation. Start from initial value, ignore Crc_StartValue32. • FALSE: Subsequent call in a call sequence. Crc_StartValue32 is interpreted to be the return value of the previous function call.
----	------------------------	--

Returns

32 bit result of CRC calculation.

6.1.5.13 Crc_GetVersionInfo()

```
void Crc_GetVersionInfo (
    Std_VersionInfoType * Versioninfo )
```

Software module version query.

Returns the version information of this module

Parameters

in	<i>Versioninfo</i>	- Pointer to get Version Information
----	--------------------	--------------------------------------

Returns

void

6.2 CRC IPL Driver

6.2.1 Detailed Description

Data Structures

- struct [Crc_Ip_CrcProtocolInfoType](#)
This type contains the CRC Protocol Information. [More...](#)
- struct [Crc_Ip_LogicChannelStateType](#)
Internal State for the Logic CHannel. [More...](#)
- struct [Crc_Ip_LogicChannelConfigType](#)
This type contains the CRC Ip Channel Configuration. [More...](#)
- struct [Crc_Ip_LogicChannelType](#)
This type contains the Crc Ip Channel Type. [More...](#)
- struct [Crc_Ip_InitType](#)
This type contains the Crc Ip Initialization. [More...](#)

Enum Reference

- enum [Crc_Ip_CrcWidthType](#)
This type contains the CRC Width Type.
- enum [Crc_Ip_CalculationType](#)
This type contains the CRC Ip Channel Configuration.
- enum [Crc_Ip_ProtocolType](#)
This type contains the CRC Ip Channel Protocol.
- enum [Crc_Ip_AlgorithmType](#)
This type contains the CRC Ip Algorithm options.

Function Reference

- void [Crc_Ip_Init](#) (const [Crc_Ip_InitType](#) *const pxCrcIpInit)
This function initializes the CRC Driver in IP Layer.
- void [Crc_Ip_SetChannelConfig](#) (const uint32 LogicChannel, const [Crc_Ip_LogicChannelConfigType](#) *pxCrcIpLogicChannelConfig)
This function initializes the CRC Channel configuration by LogicChannel's info.
- uint64 [Crc_Ip_SetChannelCalculate](#) (const uint32 LogicChannel, const uint8 *DataPtr, const uint32 Length, const uint64 StartValue, const boolean IsFirstCall)
This function shall start algorithm calculate CRC in IP layer.
- uint64 [Crc_Ip_GetChannelResult](#) (const uint32 LogicChannel)
This function get result CRC after finish calculated in IP Layer.

6.2.2 Data Structure Documentation

6.2.2.1 struct Crc_Ip_CrcProtocolInfoType

This type contains the CRC Protocol Information.

Definition at line 103 of file Crc_Ip_State.h.

6.2.2.2 struct Crc_Ip_LogicChannelStateType

Internal State for the Logic CHannel.

It shall contain the last loaded configuration by the user for the specified Logic Channel

Definition at line 120 of file Crc_Ip_State.h.

Data Fields

Crc_Ip_ProtocolType	Protocol	Protocol Type: CRC-CCITT, CRC-32, CRC-8, CRC-8-H2F
const Crc_Ip_CrcProtocolInfoType *	CrcProtocolInfo	Crc Protocol Information
uint64	CrcResult	Stores the last CRC Result.

6.2.2.3 struct Crc_Ip_LogicChannelConfigType

This type contains the CRC Ip Channel Configuration.

The Channel is identified by the following structure Configure parameter for each CRC Channel

Definition at line 195 of file Crc_Ip_Types.h.

Data Fields

Crc_Ip_ProtocolType	Protocol	CRC protocol type.
uint64	PolynomValue	Polynomial input, MSBit first. Example polynomial: 0x1021U = 1_0000_0010_0001 = $x^{12}+x^5+1$
boolean	WriteBitSwap	Swap when writing CRC input data.
boolean	WriteByteSwap	Swap when writing CRC input data.
boolean	ReadBitSwap	Swap result bits after algorithm finished calculating CRC .
boolean	ReadByteSwap	Swap result bytes after algorithm finished calculating CRC .
boolean	InverseEnable	The inversion operation is a complement (or negation) of the content.
const uint8 *	LookUpTable	LookUp Table address for 8 bits

6.2.2.4 struct Crc_Ip_LogicChannelType

This type contains the Crc Ip Channel Type.

The Crc Ip Initialization contains all the information required to initialize the CRC Driver Internal driver structure.

Definition at line 212 of file Crc_Ip_Types.h.

Data Fields

Crc_Ip_CalculationType	CalculationType	Software, Table, Hardware Calculation
uint8	HwInst	CRC hardware instance.
uint8	HwChannel	CRC hardware channel.
Crc_Ip_AlgorithmType	Algorithm	CRC algorithm type.
Crc_Ip_LogicChannelConfigType *	LogicChannelConfig	Pointer to LogicChannelConfig.

6.2.2.5 struct Crc_Ip_InitType

This type contains the Crc Ip Initialization.

The Crc Ip Initialization contains all the information required to initialize the CRC Driver Internal driver structure.

Definition at line 235 of file Crc_Ip_Types.h.

Data Fields

const Crc_Ip_LogicChannelType *const *	LogicChannelConfigList	Pointer to list LogicChannelConfig
--	------------------------	------------------------------------

6.2.3 Enum Reference

6.2.3.1 Crc_Ip_CrcWidthType

enum [Crc_Ip_CrcWidthType](#)

This type contains the CRC Width Type.

Select Width for each Channel to calculate CRC.

Definition at line 87 of file Crc_Ip_State.h.

6.2.3.2 Crc_Ip_CalculationType

enum [Crc_Ip_CalculationType](#)

This type contains the CRC Ip Channel Configuration.

The Channel is identified by the following structure Configure parameter for each CRC Channel Internal driver enumeration.

Enumerator

CRC_IP_RUNTIME_CALCULATION	Slower execution (software calculate), but small code size (no ROM table)
CRC_IP_TABLE_CALCULATION	Fast execution (software calculate), but larger code size (ROM table)
CRC_IP_HARDWARE_CALCULATION	Fast execution, less CPU time

Definition at line 129 of file Crc_Ip_Types.h.

6.2.3.3 Crc_Ip_ProtocolType

enum [Crc_Ip_ProtocolType](#)

This type contains the CRC Ip Channel Protocol.

The Protocol shall be selected from the available list. Software CRC supports all protocols. Table based CRC support the following protocols: Hardware CRC supports implementation specific protocols.

Enumerator

CRC_PROTOCOL_8BIT_CUSTOM	Generate 8-bit CUSTOM CRC with user defined algorithm
CRC_PROTOCOL_16BIT_CUSTOM	Generate 16-bit CUSTOM CRC with user defined algorithm
CRC_PROTOCOL_32BIT_CUSTOM	Generate 32-bit CUSTOM CRC with user defined algorithm
CRC_PROTOCOL_64BIT_CUSTOM	Generate 64-bit CUSTOM CRC with user defined algorithm
CRC_PROTOCOL_8BIT_H2F	Generate 8-bit CRC: H2F, AUTOSAR 4.0 CRC-8 0x2F
CRC_PROTOCOL_8BIT_SAE_J1850	Generate 8-bit CRC: VDA CAN, SAE-J1850 CRC-8.
CRC_PROTOCOL_16BIT_ARC	Generate 16-bit CRC ARC code with Polynomial: 0x8005
CRC_PROTOCOL_16BIT_CCITT_FALSE	Generate 16-bit CRC: x.25. CCITT16 CRC-16 0x1021
CRC_PROTOCOL_32BIT_E2E_P4	Generate 32-bit CRC: E2E Profile 4 CRC-32 P4 0xF4ACFB13
CRC_PROTOCOL_32BIT_ETHERNET	Generate 32-bit CRC: Ethernet (IEEE 802,3) CCITT32 CRC-32 0x04C11DB7
CRC_PROTOCOL_32BIT_CASTAGNOLI	Generate 32-bit CRC: Castagnoli
CRC_PROTOCOL_64BIT_ECMA	Generate 64-bit CRC: ECMA
CRC_PROTOCOL_INVALID	Invalid protocol

Definition at line 143 of file Crc_Ip_Types.h.

6.2.3.4 Crc_Ip_AlgorithmType

```
enum Crc_Ip_AlgorithmType
```

This type contains the CRC Ip Algorithm options.

The algorithm shall be selected from the available list. Only CRC32C support this choosing algorithm feature. Table based CRC support the following algorithm: These algorithm will calculate the CRC with different time intervals.

Enumerator

CRC_ALGORITHM_BIT_WISE	Use Bitwise algorithm for generating CRC
CRC_ALGORITHM_SLICE_BY↵ _4	Use Slice-by-4 algorithm for generating CRC. Fast execution, but larger code size
CRC_ALGORITHM_SLICE_BY↵ _8	Use Slice-by-8 algorithm for generating CRC. Fast execution, but larger code size

Definition at line 167 of file Crc_Ip_Types.h.

6.2.4 Function Reference

6.2.4.1 Crc_Ip_Init()

```
void Crc_Ip_Init (
    const Crc_Ip_InitType *const pxCrcIpInit )
```

This function initializes the CRC Driver in IP Layer.

This service is a non-reentrant function that shall initialize the Crc driver. The user must make sure that the clock is enabled.

Parameters

in	<i>pxLogicChannelConfig</i>	- Pointer to the configuration structure.
----	-----------------------------	---

Returns

void

6.2.4.2 Crc_Ip_SetChannelConfig()

```
void Crc_Ip_SetChannelConfig (
    const uint32 LogicChannel,
    const Crc_Ip_LogicChannelConfigType * pxLocLogicChannelConfig )
```

This function initializes the CRC Channel configuration by LogicChannel's info.

This service is a reentrant function that shall initialize parameters list for the CRC Channel. The list is composed of an array of Crc Channel parameters settings.

Parameters

in	<i>LogicChannel</i>	- Specifies the Logic Channel Tag defined by the user
in	<i>pxLocLogicChannelConfig</i>	- Specifies the Logic Channel Config defined by the user

Returns

void

6.2.4.3 Crc_Ip_SetChannelCalculate()

```
uint64 Crc_Ip_SetChannelCalculate (
    const uint32 LogicChannel,
    const uint8 * DataPtr,
    const uint32 Length,
    const uint64 StartValue,
    const boolean IsFirstCall )
```

This function shall start algorithm calculate CRC in IP layer.

This service is a reentrant function that shall start algorithm calculate in CRC Channel with LogicChannelConfig info

Parameters

in	<i>LogicChannel</i>	- Specifies the Logic Channel Tag defined by the user.
in	<i>*DataPtr</i>	- Pointer to start address of data block to be calculated..
in	<i>Length</i>	- Length of DataPtr block to be calculated in bytes. *
in	<i>StartValue</i>	- Start value (seed Value) when the algorithm starts.
in	<i>IsFirstCall</i>	- TRUE: ignore StartValue the initialization value is known by the chosen algorithm. • FALSE: initialization StartValue is interpreted to be the return value of the previous function

Returns

Result of CRC calculated.

6.2.4.4 Crc_Ip_GetChannelResult()

```
uint64 Crc_Ip_GetChannelResult (  
    const uint32 LogicChannel )
```

This function get result CRC after finish calculated in IP Layer.

This service is a function that shall get result from CRC Channel after finish calculated

Parameters

in	<i>LogicChannel</i>	- Specifies the Logic Channel Tag defined by the user
----	---------------------	---

Returns

result of CRC calculated.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.